Desarrollo de un prototipo de aplicación móvil de análisis automatizado de URLs extraídos de códigos QR mediante análisis aislado: Módulo de escaneo y diseño de interfaz de usuario con filtro de consulta estática a APIs de detección de amenazas

María José Villafuerte Arredondo



UVG UNIVERSIDAD DEL VALLE DE GUATEMALA

UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería



Desarrollo de un prototipo de aplicación móvil de análisis automatizado de URLs extraídos de códigos QR mediante análisis aislado: Módulo de escaneo y diseño de interfaz de usuario con filtro de consulta estática a APIs de detección de amenazas

Trabajo de graduación en modalidad de Tésis presentado por
María José Villafuerte Arredondo
Para optar al grado académico de Licenciado en Ingeniería en Ciencias de la Computación y Tecnologías de la Información

Guatemala, Noviembre del 2026

Vo.Bo.:

(f) _____

Ing. Juan Carlos Canteo

Tribunal Examinador:

(f) _____

Ing. Juan Carlos Canteo

(f) _____

MSc. Carlos Juárez

(f) _____

Ing. Luis Pedro Gonzáles

Fecha de aprobación: Guatemala, 5 de diciembre de 2018.

Prefacio

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras vitae eleifend ipsum, ut mattis nunc. Pellentesque ac hendrerit lacus. Cras sollicitudin eget sem nec luctus. Vivamus aliquet lorem id elit venenatis pellentesque. Nam id orci iaculis, rutrum ipsum vel, porttitor magna. Etiam molestie vel elit sed suscipit. Proin dui risus, scelerisque porttitor cursus ac, tempor eget turpis. Aliquam ultricies congue ligula ac ornare. Duis id purus eu ex pharetra feugiat. Vivamus ac orci arcu. Nulla id diam quis erat rhoncus hendrerit. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Sed vulputate, metus vel efficitur fringilla, orci ex ultricies augue, sit amet rhoncus ex purus ut massa. Nam pharetra ipsum consequat est blandit, sed commodo nunc scelerisque. Maecenas ut suscipit libero. Sed vel euismod tellus.

Proin elit tellus, finibus et metus et, vestibulum ullamcorper est. Nulla viverra nisl id libero sodales, a porttitor est congue. Maecenas semper, felis ut rhoncus cursus, leo magna convallis ligula, at vehicula neque quam at ipsum. Integer commodo mattis eros sit amet tristique. Cras eu maximus arcu. Morbi condimentum dignissim enim non hendrerit. Sed molestie erat sit amet porttitor sagittis. Maecenas porttitor tincidunt erat, ac lacinia lacus sodales faucibus. Integer nec laoreet massa. Proin a arcu lorem. Donec at tincidunt arcu, et sodales neque. Morbi rhoncus, ligula porta lobortis faucibus, magna diam aliquet felis, nec ultrices metus turpis et libero. Integer efficitur erat dolor, quis iaculis metus dignissim eu.

Agradecimientos

Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Sed vulputate, metus vel efficitur fringilla, orci ex ultricies augue, sit amet rhoncus ex purus ut massa. Nam pharetra ipsum consequat est blandit, sed commodo nunc scelerisque. Maecenas ut suscipit libero. Sed vel euismod tellus.

Proin elit tellus, finibus et metus et, vestibulum ullamcorper est. Nulla viverra nisl id libero sodales, a porttitor est congue. Maecenas semper, felis ut rhoncus cursus, leo magna convallis ligula, at vehicula neque quam at ipsum. Integer commodo mattis eros sit amet tristique. Cras eu maximus arcu. Morbi condimentum dignissim enim non hendrerit. Sed molestie erat sit amet porttitor sagittis. Maecenas porttitor tincidunt erat, ac lacinia lacus sodales faucibus.

xxxxxxx

Índice

Prefacio III

Agradecimientos IV

Lista de Figuras VIII

Lista de Cuadros IX

Resumen X

1. Introducción 1

2. Objetivos 2
2.1. Objetivo General . . . 2
2.2. Objetivos Específicos . . . 2

3. Justificación 3

4. Marco Teórico 6

5. Marco teórico 7
5.1. Amenazas de Seguridad en el entorno móvil . . . 7
5.1.1. Phishing: Definición, Evolución y Estado del Arte . . . 7
5.1.2. Quishing: El Phishing Mediante Códigos QR . . . 8
5.1.3. Ingeniería Social y Susceptibilidad del Usuario Móvil . . . 8
5.1.4. Análisis de URLs Maliciosas: Técnicas y Desafíos . . . 8
5.2. Estándar QR y su explotación como vector de ataque . . . 9
5.2.1. Fundamentos Técnicos del Código QR (ISO/IEC 18004) . . . 9
5.2.2. Tipos de Contenido Codificable y su Relevancia en la Detección . . . 10
5.2.3. QR como Vector de Ataque: Superficie, Motivación y Anatomía . . . 10
5.3. Sistemas de detección y clasificación de amenazas . . . 10
5.3.1. Listas Negras y Sistemas de Reputación de URLs . . . 10
5.3.2. Threat Intelligence: Plataformas Colaborativas y Enriquecimiento de Datos . 11
5.3.3. Análisis Heurístico de URLs . . . 12
5.3.4. Machine Learning Aplicado a la Detección de URLs Maliciosas . . . 12
5.3.5. Métricas de Evaluación de Clasificadores Binarios . . . 12
5.4. Escaneo y procesamiento nativo en dispositivos móviles . . . 13

5.4.1. APIs Nativas de Detección de Códigos de Barras: Apple Vision y Google ML Kit . . . 13
5.4.2. react-native-vision-camera y el Módulo CodeScanner . . . 14
5.4.3. Decodificación de QR en Tiempo Real: Latencia y Rendimiento . . . 14
5.5. Desarrollo móvil multiplataforma con React Native . . . 14
5.5.1. Arquitectura de React Native: Bridge, JSI y la New Architecture . . . 15
5.5.2. Desarrollo Cross-Platform para iOS y Android desde una Sola Base de Código 15
5.6. Seguridad en aplicaciones móviles . . . 15
5.6.1. OWASP Mobile Application Security Verification Standard (MASVS) . . . 16
5.6.2. Seguridad de Red en iOS: App Transport Security (ATS) . . . 16
5.6.3. Seguridad de Red en Android: Network Security Configuration . . . 16
5.6.4. Almacenamiento Local Seguro: MMKV . . . 17
5.7. APIs de Threat Intelligence Integradas en el Filtro F1 . . . 17
5.7.1. Google Safe Browsing API . . . 17
5.7.2. VirusTotal API . . . 18
5.7.3. PhishTank . . . 18
5.7.4. URLhaus (abuse.ch) . . . 18
5.7.5. MISP: Plataforma de Intercambio de Indicadores de Compromiso . . . 19
5.8. Geolocalización y Servicios de Contexto . . . 19
5.8.1. Geolocalización y Geocodificación Inversa en Plataformas Móviles . . . 19
5.9. Usabilidad y Seguridad Centrada en el Usuario . . . 20
5.9.1. Usable Security: Fundamentos y Modelo de Razonamiento . . . 20
5.9.2. Efectividad de Advertencias de Seguridad en Dispositivos Móviles . . . 20
5.10. Arquitectura de Backend en la Nube . . . 21
5.10.1. Arquitectura Serverless para el Backend del Filtro F1 . . . 21

6. Antecedentes **22**
6.1. Introducción al estado del arte . . . 22
6.2. Revisión de antecedentes relevantes . . . 23
6.2.1. El código QR como vector de ataque: caracterización y evidencia empírica . . 23
6.2.2. Detección de URLs maliciosas: listas negras, heurísticas y aprendizaje automático 24
6.2.3. Sistemas de reputación y plataformas colaborativas de threat intelligence . . 25
6.2.4. Soluciones comerciales de escaneo seguro de QR . . . 26
6.2.5. Usable security y efectividad de advertencias de seguridad . . . 26
6.2.6. Estándares y documentación técnica de referencia . . . 27
6.3. Comparación de los trabajos revisados . . . 27
6.4. Aportes y limitaciones de los trabajos revisados . . . 29
6.4.1. Aportes principales . . . 29
6.4.2. Limitaciones identificadas . . . 30
6.5. Fundamentación del proyecto propuesto . . . 31
6.6. Conclusión: vacíos identificados y necesidad del proyecto . . . 32

7. Alcance **33**
7.1. Definición del alcance . . . 33
7.2. Delimitación Funcional y Técnica . . . 34
7.2.1. Funcionalidades incluidas . . . 34
7.2.2. Stack tecnológico fijo . . . 34
7.3. Restricciones . . . 34
7.4. Exclusiones explícitas . . . 35

8. Metodología **36**
8.1. Enfoque y tipo de investigación . . . 36
8.2. Arquitectura de la solución . . . 36
8.2.1. Plataforma y versiones . . . 37
8.2.2. Decisión sobre las librerías de escaneo y almacenamiento . . . 38
8.3. Procedimiento de desarrollo . . . 38
8.3.1. El contrato de interfaz como instrumento . . . 39
8.4. Estrategia de validación . . . 39
8.4.1. Nivel 1: pruebas unitarias de la lógica de negocio . . . 40
8.4.2. Nivel 2: pruebas en dispositivo físico . . . 40
8.4.3. Nivel 3: pruebas de usabilidad . . . 40
8.5. Diseño de las mediciones en hardware . . . 40
8.5.1. Prueba 1.4: tiempo de escaneo y tasa de detección . . . 40
8.5.2. Protocolo de verificación en dispositivo . . . 42
8.6. Latencia del backend y política de degradación . . . 42
8.6.1. Taxonomía de errores y degradación . . . 42
8.7. Determinación del veredicto . . . 43
8.7.1. Validación de rango . . . 44
8.8. Métricas e instrumentos de evaluación . . . 44
8.9. Consideraciones éticas y de privacidad . . . 45
8.10. Limitaciones del método . . . 46

9. Conclusiones **48**

10.Recomendaciones **49**

Bibliografía **52**

Anexos **53**

A. Planos de Construcción **53**

Lista de Figuras

Lista de Cuadros

6.1. Comparación de los trabajos revisados según enfoque, aporte y limitación. 28

7.1. Delimitación Funcional y Técnica 34
7.2. Stack tecnológico fijo 34

8.1. Capas de la aplicación y su responsabilidad 37
8.2. Versiones verificadas del entorno de desarrollo 37
8.3. Bloques de desarrollo y criterio de cierre 39
8.4. Composición de la muestra de la prueba 1.4 41
8.5. Clasificación de errores según posibilidad de reintento 43
8.6. Bandas de puntaje y su fundamento 44
8.7. Métricas del proyecto y su instrumento de medición 45

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras vitae eleifend ipsum, ut mattis nunc. Pellentesque ac hendrerit lacus. Cras sollicitudin eget sem nec luctus. Vivamus aliquet lorem id elit venenatis pellentesque. Nam id orci iaculis, rutrum ipsum vel, porttitor magna. Etiam molestie vel elit sed suscipit. Proin dui risus, scelerisque porttitor cursus ac, tempor eget turpis. Aliquam ultricies congue ligula ac ornare. Duis id purus eu ex pharetra feugiat. Vivamus ac orci arcu. Nulla id diam quis erat rhoncus hendrerit. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos himenaeos. Sed vulputate, metus vel efficitur fringilla, orci ex ultricies augue, sit amet rhoncus ex purus ut massa. Nam pharetra ipsum consequat est blandit, sed commodo nunc scelerisque. Maecenas ut suscipit libero. Sed vel euismod tellus.

Proin elit tellus, finibus et metus et, vestibulum ullamcorper est. Nulla viverra nisl id libero sodales, a porttitor est congue. Maecenas semper, felis ut rhoncus cursus, leo magna convallis ligula, at vehicula neque quam at ipsum. Integer commodo mattis eros sit amet tristique. Cras eu maximus arcu. Morbi condimentum dignissim enim non hendrerit. Sed molestie erat sit amet porttitor sagittis. Maecenas porttitor tincidunt erat, ac lacinia lacus sodales faucibus. Integer nec laoreet massa. Proin a arcu lorem. Donec at tincidunt arcu, et sodales neque. Morbi rhoncus, ligula porta lobortis faucibus, magna diam aliquet felis, nec ultrices metus turpis et libero. Integer efficitur erat dolor, quis iaculis metus dignissim eu.

CAPÍTULO 1

Introducción

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Quisque eget consequat risus. Praesent a quam lacinia, consequat eros id, auctor tellus. Phasellus a dapibus arcu, vitae luctus leo. Aliquam erat volutpat. Suspendisse ac velit quam. Nullam risus nibh, lobortis vehicula elit non, pellentesque volutpat odio. Donec feugiat porta sapien gravida interdum. Cras odio nunc, lobortis sed pellentesque imperdiet, facilisis eu quam. Praesent pharetra, orci at tincidunt lacinia, neque nulla ornare lacus, ut malesuada elit risus non mi. Fusce pellentesque vitae sapien sed mollis. Curabitur viverra at nulla vitae porta. In et mauris lorem.

Vestibulum faucibus fringilla justo, eget facilisis elit convallis sit amet. Morbi nisi metus, hendrerit quis pellentesque non, faucibus at leo. Proin consectetur, est vel facilisis facilisis, arcu felis vestibulum quam, et fringilla metus neque at enim. Nunc justo mauris, egestas quis maximus eget, viverra vehicula nunc. Fusce eu nulla elementum, condimentum diam at, aliquam leo. Nullam sed sodales enim, eu imperdiet risus. Aliquam ornare augue leo, fringilla mattis nunc facilisis eget. Nam faucibus, libero a aliquet fermentum, magna arcu ultrices lacus, a placerat tortor turpis ut purus.

Integer eget ligula non metus egestas rutrum sit amet ut tellus. Aliquam vel convallis est, eu sodales leo. Proin consequat nisi at nunc malesuada gravida. Aliquam erat volutpat. Aliquam finibus interdum dignissim. Etiam feugiat hendrerit nisl, hendrerit feugiat ex malesuada in. Cras tempus eget arcu vitae congue. Ut non tristique mauris. Vivamus in mattis ipsum. Cras bibendum, enim bibendum commodo accumsan, ligula nulla porttitor ex, et pharetra eros nisl eget ex. Morbi at semper arcu. Curabitur massa sem, maximus id metus ut, molestie tempus quam. Vivamus dictum nunc vitae elit malesuada convallis. Donec ac semper turpis, non scelerisque justo. In congue risus id vulputate gravida. Nam ut mattis sapien.

CAPÍTULO 2

Objetivos

2.1. Objetivo General

Reducir el riesgo de acceso a sitios maliciosos codificados en códigos QR por usuarios móviles, mediante el desarrollo de una interfaz multiplataforma en React Native que integra el flujo completo de escaneo, presentación de resultados y consulta a las APIs de threat intelligence del primer filtro.

2.2. Objetivos Específicos

- Implementar el módulo de escaneo de códigos QR con extracción de URL en menos de 5 segundos y clasificación del tipo de contenido, mediante `react-native-vision-camera` con CodeScanner nativo en iOS y Android.
- Desarrollar el flujo de interacción principal entre el escaneo y la presentación del veredicto, mediante la implementación de la pantalla de carga con animación skeleton/shimmer y la pantalla de resultado con dos niveles de detalle (simple y extendido).
- Implementar las funcionalidades secundarias de historial offline, reporte y alerta social, mediante almacenamiento local sin cuenta de usuario, integración con el endpoint de reportes y generación de Story Card con geocodificación inversa.
- Integrar el filtro F1 de consulta a listas de reputación con tiempo de respuesta inferior a 5 segundos, mediante la conexión con las APIs de Google Safe Browsing, PhishTank y VirusTotal como primer filtro de análisis.

CAPÍTULO 3

Justificación

Los códigos QR se volvieron parte de la rutina diaria mucho más rápido de lo que avanzaron las medidas de seguridad que deberían acompañarlos. Cuando alguien escanea un código, no ve a dónde lo lleva el enlace antes de que su teléfono lo abra, y la mayoría de sistemas operativos móviles hacen esa redirección sin revisar nada. Ese desbalance es lo que vuelve tan efectivo al *quishing*: el riesgo está ahí, pero es invisible justo en el momento en que el usuario tiene que decidir. Por eso tiene sentido atacar el problema donde ocurre, es decir, en la pantalla de escaneo.

El tamaño del problema y su evolución ya se documentaron en la definición del problema, y el análisis de qué tan confiable es esa evidencia se desarrolla en los antecedentes. Lo que toca argumentar aquí es otra cosa: por qué vale la pena atenderlo ahora, por qué esta forma de resolverlo es la adecuada y a quién le sirve. Sobre lo primero bastan tres datos ya establecidos. El ataque se industrializó al integrarse a plataformas de *phishing-as-a-service* [18]. Los filtros de correo corporativo no leen los códigos QR incrustados como imagen, así que este vector simplemente los esquiva [29]. Y las apps de escaneo seguro que existen funcionan casi todas con listas negras, un enfoque que deja fuera a los dominios recién creados [36, 23, 4, 5]. Juntas, estas tres condiciones dejan abierta una ventana de exposición que hoy nadie cubre.

Por qué intervenir en el escaneo y no en otro punto

Hay una objeción obvia a esta propuesta: los navegadores móviles ya traen servicios de reputación de URLs, entonces el usuario estaría protegido sin importar con qué app escanee. Es una objeción válida y conviene responderla de frente, porque de ahí depende que este trabajo tenga sentido.

El punto es que el navegador avisa tarde. Para cuando muestra la advertencia, la petición al servidor del atacante ya salió: su dirección IP, el modelo de navegador y todo lo que venía codificado en la URL ya se enviaron, y lo que el servidor haya respondido ya llegó al teléfono. En una campaña de *quishing*, buena parte de lo que el atacante quiere lo consigue en ese primer contacto, sin que el usuario tenga que hacer nada más. Si el análisis se mete entre la lectura del código y la apertura del navegador, el control se adelanta un paso: ahí la URL ya se puede leer, pero todavía no hubo contacto con el atacante. Esa diferencia de un solo paso es la que separa una advertencia que llega tarde de una que llega a tiempo, y es la razón de que la solución viva en la app de escaneo y no más adelante en la cadena.

De esa decisión se desprenden las demás. La lectura del código se resuelve en el propio teléfono

con las APIs nativas de visión [9, 16, 33], así no hay que mandar la imagen a ningún lado y el tiempo total de respuesta se mantiene dentro de lo aceptable. El análisis arranca consultando fuentes de reputación porque son las que dan mayor certeza sobre amenazas ya conocidas gastando muy poco tiempo y cómputo [13, 17, 30]; son el primer filtro lógico. Que no alcancen para dominios nuevos es, justamente, la razón de que existan filtros después. Y se consultan varias fuentes en lugar de una sola porque sus coberturas son parciales y desparejas, sobre todo con campañas en español dirigidas a la región, donde se reporta bastante menos que en inglés.

A quién le sirve este trabajo

En lo teórico, el trabajo aporta al estudio de ataques que aprovechan interfaces poco convencionales, donde la señal que el usuario normalmente usaría para desconfiar (ver la URL) desaparece por diseño del medio. Ya está documentado que la gente cae más fácil en phishing por QR que en phishing por enlace directo, precisamente por eso [39, 21], lo que coloca el problema entre la seguridad informática y el factor humano. Además es un terreno poco explorado en América Latina, donde la tecnología se adoptó más rápido que las prácticas de seguridad que deberían venir con ella.

Para los usuarios, una interfaz que revise el enlace entre la lectura del código y la redirección atiende un riesgo que aparece en situaciones de todos los días: comercios, transporte público, oficinas de gobierno y pagos. Como el quishing le pega a cualquiera con un teléfono, sepa o no de tecnología [39], el valor de una herramienta así está en que no le exige al usuario saber nada especial para decidir bien.

Vale la pena aclarar hasta dónde llega esa afirmación. Lo que este trabajo aporta en diseño es convertir un veredicto técnico en una decisión que el usuario pueda tomar, en el momento y en el aparato donde le toca tomarla. Eso se mide con sus propios criterios: si el veredicto se entiende, si el flujo funciona bien y si cumple el tiempo de respuesta. Son criterios distintos a los que miden qué tan bien detecta el motor de análisis. Las dos cosas hacen falta y ninguna reemplaza a la otra: una pantalla muy clara que muestre un veredicto poco confiable no protege a nadie, y un veredicto acertado pero mal explicado tampoco. Lo que se sostiene aquí es que a la parte de comunicar se le ha puesto mucha menos atención que a la de detectar, y que documentarla bien tiene valor propio. El desempeño real de las fuentes de reputación se mide como parte de la evaluación y se reporta con sus limitaciones.

Para los negocios e instituciones que usan códigos QR para atender público, el trabajo ofrece una forma documentada de entender un riesgo que golpea directo la confianza en ese canal. Que alguien pegue una calcomanía encima de un QR legítimo [21] es un problema que el establecimiento no puede resolver por su cuenta; tener evidencia de cómo se puede verificar un enlace antes de abrirlo les da una base para pensar qué medidas tomar.

Para la Universidad del Valle de Guatemala, este trabajo suma en un área, la ciberseguridad aplicada, donde todavía hay poca producción académica local, y deja un antecedente para futuros proyectos de seguridad móvil, análisis de amenazas y educación digital. La falta de formación especializada en ciberseguridad es uno de los factores que más limitan la capacidad de respuesta de los países en desarrollo frente a amenazas nuevas; proyectos como este ayudan a cerrar esa brecha desde la universidad, tanto por lo que documentan como por lo que enseñan a quienes los hacen.

Para quienes integramos el equipo, el proyecto significa afianzar conocimientos en seguridad informática, desarrollo móvil multiplataforma e integración de servicios de *threat intelligence*, además de trabajar sobre un problema con implicaciones éticas reales: qué se le advierte al usuario, con cuánto detalle y con cuánta certeza son decisiones que afectan directamente su seguridad.

Para el público en general, el trabajo ayuda a dar visibilidad a una amenaza que ha crecido bastante pero que todavía suena poco. Al describir el problema y construir una solución que se

puede evaluar con criterios claros, el proyecto deja material útil para que instituciones públicas, empresas y centros educativos decidan con mejor información en temas de seguridad digital.

En resumen, este trabajo se justifica porque coinciden tres cosas: una amenaza que crece y que ya se volvió industria, un conjunto de soluciones existentes con limitaciones técnicas bien documentadas, y una población expuesta que no tiene ninguna protección en el momento que más importa. Atacar el problema desde la app móvil, el único punto donde la URL se puede leer antes de que haya contacto con el atacante, tiene sentido tanto por dónde está ubicado en la cadena del ataque como porque permite evaluar la propuesta con criterios concretos de desempeño y usabilidad.

CAPÍTULO 4

Marco Teórico

CAPÍTULO 5

Marco teórico

5.1. Amenazas de Seguridad en el entorno móvil

Este bloque contextualiza el problema de seguridad que motiva el proyecto: el phishing y su variante por códigos QR (quishing). Se revisan las técnicas de ingeniería social que hacen vulnerable al usuario móvil y los enfoques computacionales que existen para detectar URLs maliciosas, para justificar técnica y empíricamente el sistema de filtrado propuesto.

5.1.1. Phishing: Definición, Evolución y Estado del Arte

El phishing es una forma de ingeniería social en la que un atacante suplanta la identidad de una entidad de confianza para inducir a la víctima a revelar credenciales, datos financieros u otra información sensible, o a dirigirla hacia contenido malicioso. El nombre viene de la analogía con la pesca (fishing): el atacante lanza un 'anzuelo' (un mensaje o enlace falsificado) y espera a que la víctima 'muerda'. El phishing existe desde los años noventa, pero su sofisticación y volumen no han dejado de crecer, adaptándose a cada canal de comunicación nuevo que adoptan los usuarios.

El Anti-Phishing Working Group (APWG) registró más de un millón de ataques de phishing únicos en el primer trimestre de 2024, con una tendencia al alza que se mantiene en 2025 [4, 5]. El Data Breach Investigations Report de Verizon de 2024 señala al phishing como uno de los vectores de acceso inicial más frecuentes en incidentes de seguridad a nivel global [38]. Estos datos confirman que el phishing es una amenaza real y activa, no un riesgo teórico.

El prototipo de este trabajo ataca una variante específica del phishing: la que se distribuye mediante códigos QR. Entender el ecosistema del phishing, sus motivaciones, vectores, infraestructura y ciclo de vida, es necesario para diseñar bien los filtros de análisis, interpretar los resultados de las APIs de reputación y comunicar el veredicto de forma clara para el usuario final.

5.1.2. Quishing: El Phishing Mediante Códigos QR

El quishing (de QR y phishing) es el phishing que usa códigos QR para distribuir el enlace malicioso. A diferencia del phishing tradicional, donde la URL fraudulenta está a la vista en el mensaje, el quishing la esconde dentro del patrón visual del QR, así que el usuario no puede inspeccionarla directamente [18, 29].

Un ataque de quishing típico sigue cuatro pasos: se genera un QR que codifica una URL bajo control del atacante; se distribuye en un canal de alta confianza (correo corporativo, cartel físico, factura, menú de restaurante); la víctima lo escanea con la cámara del teléfono; y el navegador abre la URL sin que nadie la haya evaluado [39].

Recorded Future [18] y Palo Alto Networks Unit 42 [29] documentan el crecimiento exponencial de las campañas de quishing desde 2023, impulsado en parte por generadores de QR gratuitos y por la adopción masiva del QR en contextos de confianza (pagos, menús, accesos, verificación de identidad). Vidas et al. [39] demostraron además, con un estudio empírico, que los usuarios de smartphones caen más fácil en phishing por QR que en phishing por URL directa, precisamente porque el QR elimina la señal visual de advertencia que da una URL sospechosa.

El quishing es exactamente el problema que resuelve esta aplicación. Todo el flujo, desde el escaneo hasta la extracción de la URL, el análisis multicapa y la presentación del veredicto, responde a la anatomía de este ataque. La solución mete un punto de inspección entre el momento en que el usuario escanea el QR y el momento en que el navegador abre la URL, cerrando la ventana de vulnerabilidad que hace posible el quishing.

5.1.3. Ingeniería Social y Susceptibilidad del Usuario Móvil

La ingeniería social son las técnicas psicológicas y comunicativas con las que un atacante manipula a alguien para que revele información confidencial o haga algo que comprometa la seguridad de un sistema. A diferencia de un ataque puramente técnico, explota sesgos cognitivos humanos: urgencia, autoridad, reciprocidad, confianza. En el quishing, el código QR aprovecha la confianza del entorno físico donde aparece: un QR en el mostrador de un banco proyecta la misma autoridad que el banco mismo.

Vidas et al. [39] y Krombholz et al. [21] encontraron que el uso móvil agrava esta susceptibilidad: escanear y abrir la URL es casi un reflejo, la pantalla pequeña dificulta revisar la barra del navegador, y nadie verifica el destino antes de escanear un QR. Esto no es un simple descuido individual, sino un problema estructural que la solución técnica tiene que compensar.

Entender estos mecanismos de ingeniería social es lo que guía el diseño de la interfaz del prototipo: el momento, el tono y el formato del veredicto de seguridad tienen que contrarrestar las ganas de seguir navegando cuando el resultado indica riesgo. Este tema conecta directamente con el bloque de usable security más adelante y sustenta las decisiones de diseño de las pantallas de advertencia.

5.1.4. Análisis de URLs Maliciosas: Técnicas y Desafíos

Analizar una URL maliciosa significa aplicar técnicas computacionales para determinar si representa un riesgo de seguridad. Sahoo, Liu y Hoi [34] agrupan estas técnicas en tres categorías: las basadas en listas negras, que consultan bases de datos de URLs ya conocidas como maliciosas; las heurísticas, que evalúan rasgos de la URL como longitud, entropía, presencia de una IP literal, TLD sospechoso o edad del dominio; y las de machine learning, que extraen features de la URL y del sitio para clasificarla con modelos estadísticos o de aprendizaje profundo.

Cada categoría tiene sus propios compromisos entre cobertura, velocidad, falsos positivos y capacidad de detectar amenazas de zero-day, es decir, URLs aún no registradas en ninguna lista. Las listas negras son precisas pero tardan en incorporar sitios nuevos; los enfoques heurísticos y de ML detectan amenazas nuevas, pero a costa de más falsos positivos.

Ma et al. [23] y Le et al. [22] mostraron que los modelos basados solo en features léxicas de la URL logran alta precisión sin necesidad de visitar el sitio, algo particularmente útil en un contexto móvil donde la privacidad y la latencia son restricciones críticas. Sheng et al. [36] documentaron, por su parte, las limitaciones de depender solo de listas negras: la cobertura inicial es baja para URLs recién creadas. Esa evidencia es la que justifica el diseño multicapa del sistema, donde el Filtro F1 (listas negras) es apenas el primero de varios mecanismos.

La arquitectura de filtros del prototipo (F1 de listas de reputación, F2 de heurística de URL, F3 de sandbox y F4 de browser isolation) es una aplicación directa de esta taxonomía. En este trabajo solo se implementa completamente el F1, pero diseñar las pantallas de F2 a F4 también requiere entender los principios técnicos de cada enfoque, para representar sus resultados de forma correcta.

5.2. Estándar QR y su explotación como vector de ataque

Este bloque revisa el estándar ISO/IEC 18004 que rige los códigos QR, los tipos de contenido que pueden codificar y cómo los atacantes explotan esas capacidades para distribuir URLs de phishing. Entender la estructura del QR es un prerequisito para diseñar bien la lógica de clasificación del contenido y el punto de intervención de la herramienta.

5.2.1. Fundamentos Técnicos del Código QR (ISO/IEC 18004)

El código QR es una simbología de código de barras bidimensional estandarizada por ISO/IEC 18004 [19, 20]. A diferencia de un código de barras unidimensional, un QR puede codificar hasta 4.296 caracteres alfanuméricos o 7.089 dígitos numéricos en una cuadrícula de módulos blancos y negros. La especificación define cuatro modos de codificación (numérico, alfanumérico, byte y kanji), cuatro niveles de corrección de errores (L, M, Q, H, con redundancias del 7 % al 30 %) y cuarenta versiones de tamaño que determinan cuánta información cabe y qué tan densa es la cuadrícula.

La edición vigente, ISO/IEC 18004:2024 [20], agrega mejoras de conformidad respecto a la de 2015 pero mantiene compatibilidad hacia atrás. Esa robustez, sumada a la capacidad de meter una URL completa en un patrón compacto que cualquier cámara lee, explica tanto la adopción masiva del QR como su explotación por atacantes.

Conocer bien el estándar QR importa por dos razones concretas: primero, para saber qué tipos de dato puede llevar un QR y cuáles necesitan análisis de seguridad (solo las URLs representan riesgo de phishing; texto, WiFi o vCard no); segundo, para diseñar la lógica que bifurca el flujo de la app entre analizar una URL o mostrar una pantalla informativa cuando no lo es.

La clasificación del contenido decodificado del QR, que es el Objetivo Específico 1 de este trabajo, depende directamente de cómo ISO/IEC 18004 estructura los datos. El módulo de escaneo tiene que distinguir entre los distintos tipos de contenido para decidir si procede el análisis de seguridad o si basta con la pantalla informativa.

5.2.2. Tipos de Contenido Codificable y su Relevancia en la Detección

El estándar ISO/IEC 18004 define la codificación a nivel de bytes, pero en la práctica los generadores de QR siguen convenciones ya establecidas para tipos de contenido concretos: URL (https://, http://), texto plano, contacto vCard, credenciales WiFi (WIFI:T:WPA;S:...;P:...;;), eventos de calendario (vCalendar) y datos de pago. Cada tipo tiene un prefijo reconocible en el contenido decodificado.

No todos representan el mismo riesgo. Las URLs son el vector principal del quishing; los datos WiFi exponen credenciales de red; las vCards pueden traer contactos falsificados. Krombholz et al. [21] hicieron una taxonomía de estos vectores de ataque vía QR y confirmaron que la URL es, con diferencia, el de mayor impacto y prevalencia.

La lógica de clasificación del módulo de escaneo determina el flujo de la app: si el contenido es una URL, se activa el análisis hacia el backend; si no, se muestra la pantalla informativa. Esta bifurcación es uno de los componentes centrales del Objetivo Específico 1.

5.2.3. QR como Vector de Ataque: Superficie, Motivación y Anatomía

La superficie de ataque del quishing es amplia porque los códigos QR aparecen en entornos de mucha confianza: facturas corporativas, señalética de comercios, materiales de marketing, correos institucionales, pantallas de autenticación en dos pasos. Un atacante puede reemplazar un QR legítimo con una calcomanía encima del original, o distribuir uno malicioso digitalmente en una campaña de spear phishing. La motivación principal es evadir los filtros de URL del correo y de los proxies corporativos, que revisan el texto del mensaje pero no lo que hay codificado dentro de una imagen.

Palo Alto Networks Unit 42 [29] documentó que plataformas como Microsoft 365 y Google Workspace no escanean automáticamente el contenido de los QR embebidos en correos, lo que convierte al quishing en una técnica de evasión efectiva. Recorded Future [18] reportó un aumento del 587 % en ataques de quishing entre el segundo semestre de 2023 y el primero de 2024.

Entender la anatomía del ataque y cómo se distribuye es clave para ubicar la propuesta de valor del prototipo: la aplicación interviene justo en el único punto del ciclo de ataque que todavía no tiene protección automatizada, el momento del escaneo desde el teléfono del usuario.

El diseño del flujo, y en particular la decisión de interceptar la navegación antes de que el sistema operativo abra el navegador, responde directamente a esta mecánica del ataque. Sin entenderla, no se podría justificar por qué la herramienta se inserta ahí y no en otro punto del ciclo.

5.3. Sistemas de detección y clasificación de amenazas

Este bloque presenta las categorías de técnicas para identificar URLs maliciosas (listas negras, análisis heurístico y aprendizaje automático), las plataformas colaborativas de threat intelligence que las alimentan y las métricas para evaluar qué tan bien funcionan. Este marco técnico es la base de la arquitectura de filtros multicapa del prototipo.

5.3.1. Listas Negras y Sistemas de Reputación de URLs

Una lista negra de URLs es una base de datos con URLs, dominios o direcciones IP identificadas como maliciosas, fraudulentas o asociadas a phishing, malware o spam. Los sistemas de reputación van un paso más allá: le asignan a cada URL o dominio una puntuación derivada de varias señales,

como el historial de actividad maliciosa, la asociación con infraestructura de amenazas conocida, las características del registro del dominio y el análisis de contenido.

Sheng et al. [36] evaluaron la cobertura y latencia de varias blacklists en 2009 y encontraron que las más grandes cubrían apenas el 20 % de los sitios de phishing activos al momento en que la URL se creó, con la cobertura mejorando conforme el sitio se mantenía activo y era denunciado. De ahí sale el concepto de zero-hour phishing como límite estructural de este enfoque.

Vale la pena aclarar que ese estudio es de 2009, cuando los sistemas de reputación eran mucho menos maduros. Hoy, plataformas como Google Safe Browsing actualizan sus bases cada pocos minutos con análisis automatizado a escala masiva. Aun así, los informes de APWG de 2024 y 2025 [4, 5] muestran que el zero-hour phishing, sitios maliciosos activos por menos de una hora antes de ser indexados, sigue siendo una fracción importante de los ataques. Eso confirma que la limitación estructural de las listas negras persiste sin importar qué tan rápido se actualicen, y refuerza el argumento a favor de un diseño multicapa.

Las listas de reputación son el Filtro F1 del prototipo, el primero y más rápido de los mecanismos de análisis. Su eficiencia (responden en milisegundos para URLs ya conocidas), su cobertura acumulada y su bajo costo computacional las hacen el punto de decisión inicial ideal en un sistema que necesita responder en tiempo real.

El Objetivo Específico 4 fija un tiempo de respuesta del Filtro F1 inferior a 5 segundos. Las tres APIs elegidas (Google Safe Browsing [13], PhishTank [30] y VirusTotal [17]) representan el estado del arte en listas de reputación, y su integración en el backend es el componente funcional central de este trabajo de graduación.

5.3.2. Threat Intelligence: Plataformas Colaborativas y Enriquecimiento de Datos

La threat intelligence es la disciplina de recolectar, analizar y compartir información sobre amenazas de seguridad, como actores, tácticas, técnicas e indicadores de compromiso (IOC), con el fin de anticipar o responder a ataques. Para detectar URLs maliciosas, los IOC relevantes son dominios, URLs, direcciones IP y hashes de archivos ligados a campañas de phishing, malware o infraestructura de comando y control.

Las plataformas colaborativas de threat intelligence parten de una idea simple: ninguna organización ve sola el panorama completo de amenazas, así que al compartir datos de incidentes la comunidad construye una inteligencia colectiva más robusta. MISP [26] y URLhaus [1] son ejemplos de plataformas abiertas para esto. VirusTotal [17] hace algo parecido pero distinto: agrega los resultados de más de 70 motores antivirus, funcionando más como plataforma de enriquecimiento de datos.

Que las fuentes que alimentan el Filtro F1 sean colaborativas es clave en el diseño: cada vez que un usuario reporta un QR malicioso desde la app (Feature 5.2), esa información puede retroalimentar blacklists como PhishTank o URLhaus, convirtiendo a los usuarios del prototipo en participantes activos del ecosistema de threat intelligence.

La funcionalidad de reporte de QR malicioso (Objetivo Específico 3) implementa justamente este modelo colaborativo: el prototipo no solo consume inteligencia de amenazas, también la genera y la aporta al enviar URLs reportadas a las blacklists correspondientes.

5.3.3. Análisis Heurístico de URLs

El análisis heurístico evalúa la estructura léxica y semántica de una URL para estimar si es maliciosa, sin consultar bases externas ni visitar el sitio. Las heurísticas más usadas incluyen: longitud anómala, presencia de una IP literal en vez de un dominio, subdominios excesivos o engañosos (typosquatting), TLDs poco convencionales, caracteres especiales y patrones de ofuscación, uso de acortadores, ausencia de HTTPS, y la edad del dominio, ya que uno recién registrado es más sospechoso.

Le et al. [22] desarrollaron PhishDef, un clasificador basado exclusivamente en características léxicas de la URL con precisión superior al 95 %. Ma et al. [23] llegaron más lejos: mostraron que usando solo el nombre del dominio se puede alcanzar 99 % de precisión para phishing conocido, sin necesidad de acceder al contenido del sitio.

El Filtro F2 del prototipo se basa en este tipo de análisis. Su backend no se implementa en este trabajo de graduación, pero la pantalla de resultados, que muestra el desglose de criterios activados (edad del dominio, TLD sospechoso, estructura de la URL, HTTPS), sí necesita una comprensión precisa de qué evalúa cada heurística, para que la representación visual sea técnicamente correcta.

Esa pantalla, con su score de riesgo y desglose de criterios, es simplemente una interfaz para los resultados del análisis heurístico. Los criterios que muestra corresponden directamente a las features documentadas en la literatura [23, 22], así que la interfaz refleja con fidelidad lo que produce el análisis técnico.

5.3.4. Machine Learning Aplicado a la Detección de URLs Maliciosas

Los enfoques de machine learning tratan la detección de URLs maliciosas como clasificación binaria (benigna o maliciosa) o multiclase (phishing, malware, spam, benigna). Cada URL se representa con un vector de features que puede incluir rasgos léxicos, datos de WHOIS, contenido HTML de la página y registros DNS. Los algoritmos más usados en la literatura van desde Random Forest y Gradient Boosting hasta Support Vector Machines y, más recientemente, redes recurrentes y transformadores de texto [34, 23, 22].

Este trabajo no entrena ni despliega modelos propios, pero el análisis heurístico del F2 es, en su versión más avanzada, justamente un modelo de clasificación basado en features de la URL. Entender los principios de ML aplicados a este dominio permite dimensionar bien qué puede y qué no puede el F2, y justificar por qué un sistema multicapa que combina listas negras, heurística y sandbox cubre más que cualquier técnica sola.

En fases futuras, el F2 podría pasar de un conjunto de reglas heurísticas a un modelo de ML entrenado sobre datasets de URLs etiquetadas. Los más usados en la literatura son el ISCX-URL-2016 del Canadian Institute for Cybersecurity, con más de 30.000 URLs en cinco categorías (benignas, spam, phishing, malware, defacement), y el PHIUSIIL Phishing URL Dataset del UCI Machine Learning Repository, con más de 235.000 instancias etiquetadas. Conocer estos recursos orienta tanto la evaluación empírica del Filtro F1 como el diseño futuro del F2.

5.3.5. Métricas de Evaluación de Clasificadores Binarios

Evaluar un clasificador binario, como el que decide si una URL es maliciosa o no, requiere métricas más finas que la simple exactitud, sobre todo cuando las clases están desbalanceadas: las URLs maliciosas son una fracción pequeña del total. Las métricas principales son precisión, exhaustividad o recall, especificidad, F1-score y el área bajo la curva ROC. Sokolova y Lapalme [37] hacen un análisis sistemático de estas métricas y de cuándo conviene usar cada una.

En seguridad, el costo de equivocarse no es simétrico: un falso negativo, es decir no detectar una URL maliciosa, tiene consecuencias más graves para el usuario que un falso positivo que bloquea una URL legítima. Eso empuja las decisiones de umbral hacia maximizar el recall aunque se sacrifique algo de precisión.

Para evaluar el Filtro F1 de este trabajo, y para poder interpretar bien lo que reportan las APIs integradas, hace falta entender qué significan sus tasas de detección y cuáles son sus limitaciones documentadas.

Estas métricas son el marco conceptual para juzgar si el Filtro F1 cumple los objetivos de calidad del prototipo. El veredicto que ve el usuario (SEGURO, SOSPECHOSO o MALICIOSO) implica un umbral de clasificación cuyas consecuencias, falso positivo o falso negativo, tienen que estar fundamentadas en este marco teórico.

El protocolo de validación del Filtro F1 consiste en tomar un conjunto de URLs de prueba, sacadas de fuentes públicas como el feed de PhishTank, el dataset de OpenPhish [27] o el ISCX-URL-2016, y pasarlas por las tres APIs integradas (Google Safe Browsing, PhishTank y VirusTotal), comparando lo que devuelven contra la etiqueta real de cada URL. Con eso se calculan precisión, exhaustividad y F1-score del sistema completo, dando evidencia empírica sobre qué tan bien funciona y qué tasas de error tiene en los escenarios críticos.

5.4. Escaneo y procesamiento nativo en dispositivos móviles

Este bloque describe las APIs nativas de visión por computadora que decodifican códigos QR directamente en el dispositivo, sin depender de servicios externos; la biblioteca react-native-vision-camera que las envuelve para React Native; y las consideraciones de latencia que definen cuánto tiempo queda disponible para el análisis de seguridad.

5.4.1. APIs Nativas de Detección de Códigos de Barras: Apple Vision y Google ML Kit

Los sistemas operativos modernos traen APIs de visión por computadora a nivel de sistema para detectar y decodificar códigos de barras y QR directamente en el hardware del dispositivo, sin nube de por medio. En iOS, el framework Vision expone VNDetectBarcodesRequest [9], que usa el motor de visión de Apple para detectar QR, Aztec, PDF417 y otros formatos, con aceleración por hardware vía el Neural Engine.

En Android, Google ML Kit Barcode Scanning [16] hace lo mismo mediante el paquete com.google.mlkit:barcode-scanning, que opera completamente on-device, sin necesidad de conectividad, con soporte para más de diez simbologías incluyendo QR, Data Matrix y EAN.

Elegir estas APIs nativas para el módulo de escaneo, en vez de una solución de terceros, garantiza el menor overhead de procesamiento, la mejor compatibilidad con el hardware de cada plataforma y el cumplimiento de las políticas de privacidad de iOS y Android, que restringen el acceso de terceros a los frames de la cámara. El Objetivo Específico 1 pide una latencia de escaneo inferior a 5 segundos, algo que solo es alcanzable procesando en el dispositivo.

react-native-vision-camera [33], la biblioteca elegida para el módulo de escaneo, usa exactamente estas APIs como backend: VNDetectBarcodesRequest en iOS y ML Kit en Android. Conocer las capacidades y límites de estas APIs es la base de las decisiones de implementación del módulo de cámara.

5.4.2. react-native-vision-camera y el Módulo CodeScanner

react-native-vision-camera [33] es una biblioteca de código abierto de Marc Rousavy que da acceso completo a la cámara desde React Native, con soporte para procesar frames a alta velocidad mediante frame processors escritos en JavaScript o en código nativo (Swift/Kotlin) vía JSI (JavaScript Interface). Su módulo CodeScanner es una capa sobre las APIs nativas de detección de códigos de barras de iOS y Android, pensada para detectar en tiempo real sobre el stream de video de la cámara.

El módulo decodifica múltiples simbologías en tiempo real a velocidades de hasta 60 fps y notifica al componente React Native mediante un callback cada vez que detecta y decodifica un código, junto con metadatos sobre el tipo de simbología y su posición en el frame.

Se eligió esta biblioteca para el Objetivo Específico 1 por ventajas concretas frente a alternativas como react-native-camera (ya deprecated) o expo-camera: mejor rendimiento al usar JSI en vez del bridge clásico, acceso directo a las APIs nativas de escaneo, compatibilidad con la New Architecture de React Native [25] y mantenimiento activo del proyecto.

react-native-vision-camera con CodeScanner es, en el prototipo, el componente responsable de adquirir y decodificar el QR. Entender su funcionamiento (ciclo de vida, manejo de permisos, configuración del scanner, extracción del payload) es prerequisito directo para implementar el Objetivo Específico 1.

5.4.3. Decodificación de QR en Tiempo Real: Latencia y Rendimiento

La latencia de decodificación es el tiempo entre que el QR entra al campo de visión de la cámara y el sistema extrae el payload. Depende de la velocidad del sensor, la resolución del frame, la versión y densidad del QR, la iluminación, el cómputo disponible en el dispositivo y qué tan eficiente es el decodificador.

En una app de análisis de seguridad en tiempo real, esta latencia es apenas el primer tramo del presupuesto total (decodificación, transmisión al backend, consultas a APIs, retorno de resultados). El Objetivo Específico 1 pide menos de 5 segundos para el ciclo completo, así que la decodificación tiene que resolverse en fracciones de segundo.

Usar las APIs nativas de visión a través de react-native-vision-camera garantiza tiempos de decodificación por debajo de 100 ms en hardware moderno, dejando el resto del margen para el análisis de red. Esto es lo que informa la duración de la pantalla de carga con animación skeleton o shimmer (Objetivo Específico 2).

Esa pantalla de carga asume, implícitamente, un modelo de latencia: la animación tiene que durar lo suficiente para cubrir el análisis real sin dar sensación de lentitud, pero sin tentar al usuario a cancelar antes de ver el resultado.

5.5. Desarrollo móvil multiplataforma con React Native

Este bloque justifica la elección de React Native, describe cómo evolucionó su arquitectura interna, del bridge clásico a la New Architecture con JSI y Fabric, y repasa las implicaciones técnicas y de seguridad de tener una sola base de código para iOS y Android.

5.5.1. Arquitectura de React Native: Bridge, JSI y la New Architecture

React Native permite escribir componentes de interfaz en JavaScript o TypeScript que se renderizan como componentes nativos de iOS y Android. La arquitectura de React Native usaba serialización JSON para comunicar el hilo de JavaScript con los módulos nativos, lo que añadía latencia y limitaba las operaciones de alta frecuencia.

La New Architecture [25], estable desde la versión 0.71 y activa por defecto desde la 0.76, cambia ese esquema por tres piezas: JSI, que permite llamadas síncronas entre JavaScript y código nativo; Fabric, el motor de renderizado que permite componer la UI en varios hilos; y TurboModules, que reemplazan a los Native Modules con carga diferida y tipado fuerte. Esta arquitectura es requisito para usar react-native-vision-camera v4.

El prototipo se construye sobre la New Architecture, lo que habilita los frame processors de react-native-vision-camera y da el rendimiento necesario para procesar en tiempo real el stream de la cámara. Entender esta arquitectura es indispensable para razonar sobre rendimiento, límites del sistema y cómo depurar problemas.

La pila tecnológica del prototipo (React Native con New Architecture, react-native-vision-camera v4, react-native-mmkv) solo tiene sentido completo en el contexto de lo que habilita la New Architecture, que es justo lo que da esta sección.

5.5.2. Desarrollo Cross-Platform para iOS y Android desde una Sola Base de Código

El desarrollo cross-platform busca reutilizar la mayor cantidad de código posible entre plataformas distintas (iOS, Android, web) mediante abstracciones sobre las APIs nativas. React Native lo hace compilando componentes JavaScript a vistas nativas de cada plataforma, a diferencia de Flutter, que dibuja en su propio canvas, o de una PWA.

Las implicaciones de seguridad de este enfoque no son menores: las políticas de privacidad, los permisos, el almacenamiento y las restricciones de red de iOS y Android difieren bastante entre sí. El prototipo adopta las políticas de Apple como el estándar más estricto, para que cumplir en iOS implique automáticamente cumplir en Android.

Elegir React Native en vez de desarrollo nativo independiente (Swift más Kotlin) es una decisión de arquitectura con consecuencias en rendimiento, mantenibilidad, acceso a APIs nativas y ciclos de publicación, y por eso vale la pena documentarla aquí.

Que el prototipo lo desarrolle una sola persona para dos plataformas depende directamente de la productividad que da este enfoque cross-platform. Los desafíos que trae, diferencias de permisos, comportamiento de la cámara, políticas distintas entre App Store y Google Play, también son parte del contexto técnico que hay que tener claro.

5.6. Seguridad en aplicaciones móviles

Este bloque presenta el marco normativo y los mecanismos técnicos que hacen que el prototipo sea, en sí mismo, una app segura: el estándar MASVS de OWASP, las políticas de red de iOS y Android, y las opciones de almacenamiento local cifrado.

5.6.1. OWASP Mobile Application Security Verification Standard (MASVS)

El MASVS [28] es el estándar de OWASP que define los requisitos de seguridad que debe cumplir una app móvil. La versión 2.1, de 2024, organiza los controles en cinco categorías: MASVS-STORAGE (almacenamiento seguro), MASVS-CRYPTO (uso correcto de criptografía), MASVS-AUTH (autenticación y autorización), MASVS-NETWORK (seguridad de las comunicaciones) y MASVS-PLATFORM (interacción segura con el sistema operativo y otras apps). Cada categoría tiene dos niveles de madurez: L1 para todas las apps y L2 para las que manejan datos sensibles.

Una app de seguridad que promete proteger al usuario tiene que ser, ella misma, segura. Las categorías más relevantes para el prototipo son MASVS-NETWORK (comunicaciones TLS con las APIs de threat intelligence), MASVS-STORAGE (historial de escaneos guardado localmente) y MASVS-PLATFORM (permisos de cámara y geolocalización).

El MASVS es el marco normativo detrás de las decisiones de seguridad del prototipo: HTTPS en todas las comunicaciones, almacenamiento cifrado del historial local y mínimo privilegio al pedir permisos. Cada una de estas decisiones se justifica directamente contra un control del estándar.

Proteger las credenciales de las APIs externas del Filtro F1, es decir las API keys de Google Safe Browsing y VirusTotal, es un requisito de MASVS-STORAGE-2, que prohíbe guardar credenciales en texto plano en el código o en almacenamiento sin cifrar. En el prototipo, estas credenciales viven solo en el entorno de ejecución del backend serverless (variables de entorno de AWS Lambda), nunca en el cliente móvil, eliminando el riesgo de que alguien las extraiga haciendo ingeniería inversa a la app. MASVS-NETWORK-2, por su parte, exige validar el certificado TLS del servidor en cada conexión. El certificate pinning, que fija en el código de la app el certificado o la clave pública esperada del backend, impide ataques de intermediario incluso si el atacante presenta un certificado firmado por una autoridad de confianza del sistema, y conviene evaluarlo para las comunicaciones entre la app y el backend propio del prototipo.

5.6.2. Seguridad de Red en iOS: App Transport Security (ATS)

App Transport Security (ATS) [7] es el mecanismo de Apple, desde iOS 9, que exige por defecto HTTPS con TLS 1.2 o superior, certificados válidos y cifrado fuerte en toda conexión de red. Cualquier excepción se tiene que declarar explícitamente en el Info.plist de la app, y Apple la revisa durante el proceso de aprobación en la App Store. Apple Platform Security [10] documenta en detalle el modelo de seguridad de iOS, incluyendo aislamiento de procesos, firma de código y sandboxing.

Las comunicaciones del prototipo con las APIs de threat intelligence (Google Safe Browsing, VirusTotal, PhishTank) y con el backend en AWS tienen que cumplir con ATS. Entender ATS es indispensable para diseñar bien la capa de red del prototipo en iOS.

Todos los endpoints integrados en el Filtro F1 usan HTTPS, así que cumplen con ATS sin necesidad de excepciones. El único caso especial son las URLs de usuario que pueden venir en HTTP, el payload del QR: esa URL se envía al backend para analizarla, nunca se abre directamente en la app, con lo que se evita tener que declarar exenciones de ATS.

5.6.3. Seguridad de Red en Android: Network Security Configuration

Android tiene su propio equivalente a ATS: Network Security Configuration [14], un archivo XML donde se define qué autoridades certificadoras se aceptan, si se permite tráfico en texto plano (HTTP) y cómo se tratan los certificados de depuración. Desde Android 9 (API 28), el tráfico en

texto plano viene deshabilitado por defecto.

Esta configuración complementa a ATS en el diseño de la capa de red del prototipo, para que las comunicaciones sean seguras en ambas plataformas por igual. Mantener esta consistencia es parte del enfoque de usar las políticas de Apple como estándar de cumplimiento en todo el proyecto.

La configuración de red en Android tiene que ser coherente con la de iOS: todo hacia las APIs y el backend debe ir por HTTPS, sin exenciones innecesarias de texto plano. Esto se traduce directamente en cómo se arma el archivo res/xml/network_security_config.xml del proyecto.

5.6.4. Almacenamiento Local Seguro: MMKV

react-native-mmkv [31] es una biblioteca de almacenamiento clave-valor para React Native, basada en el framework MMKV de WeChat, con lectura y escritura síncronas, cifrado AES integrado y un rendimiento muy superior al de AsyncStorage, el almacenamiento por defecto de React Native. Benchmarks del propio autor [32] muestran que MMKV puede ser hasta 30 veces más rápido que AsyncStorage en lectura, con soporte para cifrado nativo en iOS (Keychain) y Android (KeyStore).

El historial de escaneos del prototipo (Objetivo Específico 3) se guarda localmente en el dispositivo, sin cuenta de usuario. Como ese historial incluye URLs escaneadas, algunas maliciosas, hay que guardarlo de forma segura y accesible sin conexión. La combinación de velocidad y cifrado nativo hace de MMKV la opción más adecuada, en línea con los controles de MASVS-STORAGE.

MMKV es, entonces, la tecnología elegida para el historial offline del prototipo. Su justificación técnica, rendimiento, cifrado y compatibilidad con la New Architecture, respalda las decisiones de implementación del Objetivo Específico 3.

5.7. APIs de Threat Intelligence Integradas en el Filtro F1

Este bloque detalla cada fuente de datos de reputación y amenazas que compone el Filtro F1: Google Safe Browsing, VirusTotal, PhishTank, URLhaus y MISP. Se describen sus capacidades técnicas, sus limitaciones operacionales y el rol que cumple cada una dentro del análisis multicapa.

5.7.1. Google Safe Browsing API

La Google Safe Browsing API [13] permite consultar si una URL aparece en las listas de amenazas de Google, que cubren phishing, malware, software no deseado e ingeniería social. La versión 5, la Safe Browsing Lookup API v5, resuelve consultas de URLs individuales en tiempo real, con una latencia típica por debajo de 200 ms. Google actualiza estas listas de forma continua con análisis automatizado a escala masiva.

Es, por volumen de URLs indexadas, la base de datos de URLs maliciosas más grande del mundo, y la misma que usan Chrome, Safari y Firefox para mostrar advertencias de sitios peligrosos. Integrarla como primera consulta del Filtro F1 extiende al contexto móvil del QR una cobertura que ya existe en el navegador de escritorio.

Es una de las tres APIs del Filtro F1, y su configuración, autenticación mediante API key, formato de solicitud y estructura de respuesta son parte directa de la implementación del Objetivo Específico 4.

5.7.2. VirusTotal API

VirusTotal [17] analiza archivos, URLs y dominios agregando los resultados de más de 70 motores antivirus y herramientas de análisis. Su API v3 permite enviar una URL y recuperar el veredicto agregado de todos los motores, además de metadatos del dominio como WHOIS, DNS y certificados TLS. El plan gratuito permite hasta 4 consultas por minuto y 500 al día.

Su fortaleza está en la agregación: una URL que se le escapa a un motor puede ser detectada por otro, y el veredicto final combina múltiples perspectivas independientes, lo que reduce la probabilidad de falsos negativos frente a depender de una sola fuente.

Es la tercera API del Filtro F1. Sus resultados, incluyendo cuántos motores marcaron la URL como maliciosa y qué tipo de amenaza identificaron, alimentan el detalle extendido que el prototipo puede mostrar en la pantalla de resultado avanzado.

5.7.3. PhishTank

PhishTank [30], operado por Cisco Talos, es una base de datos colaborativa de URLs de phishing construida a partir de reportes de usuarios verificados: cualquiera puede reportar una URL, la comunidad vota para confirmarla o rechazarla, y las confirmadas pasan a la base pública. Su API gratuita permite consultar si una URL está en esa base, que se actualiza cada hora.

PhishTank complementa a Google Safe Browsing con cobertura especializada en phishing, incluyendo campañas recientes que aún no aparecen en los sistemas automáticos de Google. Su modelo colaborativo encaja bien con el ecosistema de threat intelligence que el prototipo busca alimentar mediante el reporte de QR maliciosos.

Es la segunda API del Filtro F1. Cuando un usuario reporta un QR malicioso desde el prototipo (Feature 5.2), el backend puede enviar esa URL a PhishTank para que la comunidad la verifique, cerrando el ciclo de retroalimentación de threat intelligence.

Vale la pena una nota de estado operacional: desde finales de 2023, Cisco Talos redujo bastante su soporte activo a PhishTank, lo que ha causado interrupciones periódicas del servicio y actualizaciones menos frecuentes. Por eso la implementación del Filtro F1 tiene que manejar explícitamente errores y timeouts cuando esta API no responde. OpenPhish [27] es una alternativa activa y mantenida que cataloga URLs de phishing con actualizaciones en tiempo real, y puede servir como fuente complementaria o de respaldo.

5.7.4. URLhaus (abuse.ch)

URLhaus [1], de abuse.ch, se especializa en rastrear URLs usadas para distribuir malware. A diferencia de PhishTank, enfocado en phishing, URLhaus cataloga URLs que sirven exploits, ransomware, botnets y otros tipos de malware. Su API pública permite consultar URLs y dominios contra esa base, actualizada en tiempo real a partir de reportes de la comunidad de investigación en seguridad.

Los códigos QR no solo distribuyen phishing: también pueden llevar a páginas que sirven malware directamente. URLhaus extiende la cobertura del Filtro F1 más allá del phishing, hacia ese tipo de amenaza.

URLhaus queda fuera del alcance de este trabajo, que se limita a las tres APIs ya descritas. Se incluye en el marco teórico porque el QR no distribuye solo phishing, y URLhaus es la fuente más especializada para ese otro vector. Integrarla como cuarta fuente del Filtro F1 es una línea natural

de trabajo futuro, sobre todo pensando en quishing corporativo donde se distribuye ransomware mediante QR embebidos en documentos de empresa.

5.7.5. MISP: Plataforma de Intercambio de Indicadores de Compromiso

MISP [26] es una plataforma abierta para compartir, almacenar y correlacionar indicadores de compromiso (IOC) de incidentes de ciberseguridad. Permite a organizaciones y comunidades compartir información estructurada sobre amenazas en un formato estándar, con soporte para taxonomías, galerías de malware, feeds de threat intelligence y correlación automática de IOC entre eventos. Su arquitectura federada deja que varias instancias de MISP compartan datos de forma controlada entre comunidades de confianza.

MISP representa el estado del arte en infraestructura de threat intelligence colaborativa, y le importa a este proyecto por dos razones: primero, como contexto del ecosistema donde operan las APIs integradas, muchas de las cuales consumen feeds compatibles con MISP; segundo, como modelo posible para una futura evolución del backend hacia una plataforma propia de intercambio de IOC de quishing.

MISP no se integra directamente en el prototipo, pero la funcionalidad de reporte de QR malicioso (Feature 5.2) implementa su principio central a escala reducida: los usuarios aportan IOC, es decir URLs maliciosas, que se redistribuyen al ecosistema de threat intelligence.

Además, varias de las APIs integradas en el Filtro F1 consumen o generan datos en formatos compatibles con MISP: VirusTotal exporta sus resultados en MISP format, y PhishTank genera feeds que múltiples instancias MISP importan automáticamente. Por eso entender MISP como infraestructura subyacente es necesario para comprender del todo cómo operan las fuentes del F1 dentro del panorama más amplio del intercambio de inteligencia sobre amenazas.

5.8. Geolocalización y Servicios de Contexto

Este bloque cubre la localización geográfica del dispositivo y la geocodificación inversa, funcionalidades auxiliares que el prototipo usa para enriquecer el registro de escaneos con el nombre del lugar donde se capturó el QR, y las implicaciones de privacidad que eso trae.

5.8.1. Geolocalización y Geocodificación Inversa en Plataformas Móviles

Los teléfonos modernos determinan la ubicación combinando señales de GPS, Wi-Fi, Bluetooth y torres celulares, gestionadas por los frameworks de localización del sistema operativo: Core Location en iOS [8] y la API de localización de Android [15]. Ambas plataformas piden consentimiento explícito del usuario para acceder a la ubicación, distinguiendo entre uso en primer y segundo plano, y entre precisión aproximada y precisa.

La geocodificación inversa convierte coordenadas (latitud, longitud) en una descripción legible: nombre del establecimiento, dirección, ciudad, país. En iOS, CLGeocoder [6] hace esto consultando los servidores de Apple Maps; Mapbox [24] ofrece una API REST alternativa con más granularidad y soporte para nombres de lugares de interés.

La Story Card del prototipo (Feature 5.3, una funcionalidad auxiliar al objetivo central de detección) usa geocodificación inversa para mostrar el nombre del lugar donde se escaneó el QR en la tarjeta compartible, por ejemplo 'Restaurante El Portal, Ciudad de Guatemala'. Esto requiere acceso a la ubicación en el momento del escaneo y una geocodificación lo bastante precisa para distinguir

un establecimiento específico dentro de una misma dirección. Elegir entre CLGeocoder y Mapbox [24] afecta la privacidad, ya que los datos de ubicación viajan a servidores de terceros, además de la precisión del resultado y el costo del servicio.

El Objetivo Específico 3 incluye esta geocodificación inversa para la Story Card. Pedir permisos de localización, acceder a las coordenadas GPS e invocar el servicio de geocodificación son componentes directos de esta funcionalidad, y su implementación correcta depende de conocer bien las APIs de localización documentadas en [8, 15].

5.9. Usabilidad y Seguridad Centrada en el Usuario

Este bloque trata los principios de diseño de sistemas de seguridad que los usuarios realmente puedan usar, con énfasis en qué tan efectivas son las advertencias de seguridad en dispositivos móviles. Estos fundamentos informan directamente el diseño de la interfaz de resultados del prototipo.

5.9.1. Usable Security: Fundamentos y Modelo de Razonamiento

Usable security es la subdisciplina de la interacción persona-computadora que estudia cómo diseñar sistemas de seguridad que la gente pueda usar de forma efectiva, eficiente y satisfactoria. Cranor [11] propuso un marco donde el comportamiento seguro depende de tres pasos encadenados: el usuario tiene que notar la señal de seguridad, entender qué significa y actuar en consecuencia. Si el sistema falla en cualquiera de los tres, es inefectivo sin importar qué tan sólido sea técnicamente.

En el quishing, esto es crítico: si el prototipo presenta el veredicto de forma confusa, tarde o demasiado técnica, el usuario puede ignorarlo o malinterpretarlo, y ahí se pierde todo el beneficio de seguridad de la herramienta.

Las decisiones de interfaz del prototipo, como la paleta de colores del veredicto (verde, amarillo, rojo), la vista en dos niveles de detalle (simple y extendida), el texto del veredicto ('SEGURO', 'SOSPECHOSO', 'MALICIOSO') o la posición y tamaño de los botones de acción, buscan justamente facilitar ese comportamiento seguro según el marco de Cranor [11]. Sin esta base teórica, serían decisiones puramente estéticas.

El Objetivo Específico 2, o sea el flujo entre el escaneo y la presentación del veredicto, es en el fondo un problema de usable security. La pantalla de resultado busca maximizar la probabilidad de que el usuario note, entienda y actúe bien ante el veredicto, aplicando los principios documentados en [11, 12, 2].

5.9.2. Efectividad de Advertencias de Seguridad en Dispositivos Móviles

Una advertencia de seguridad es efectiva en la medida en que logra cambiar el comportamiento del usuario hacia algo más seguro. Egelman, Cranor y Hong [12] estudiaron empíricamente qué hace efectiva una advertencia de phishing en el navegador, y encontraron que funcionan mejor cuando interrumpen la tarea del usuario, son visualmente prominentes, usan lenguaje concreto en vez de técnico y aparecen en el momento justo. Akhawe y Felt [2] llevaron esto a datos reales de Chrome a gran escala: el 70 % de los usuarios avanza a pesar de las advertencias de SSL, pero solo el 25 % ignora las de malware, lo que muestra que el tipo y la severidad percibida de la amenaza cambian cómo reacciona la gente.

El prototipo es, en el fondo, un sistema de advertencias de seguridad en tiempo real, y esta literatura da principios de diseño basados en evidencia para sus pantallas de resultado: cuándo

interrumpir el flujo, cómo describir la amenaza, qué opciones dar y cómo formular la llamada a la acción.

Los dos niveles de detalle de la pantalla de resultado (Objetivo Específico 2), vista simple para el usuario casual y vista extendida para el técnico, aplican el principio de divulgación progresiva de la literatura de usable security: la advertencia principal tiene que entenderse sola, y los detalles técnicos quedan disponibles para quien los necesite, sin saturar al usuario promedio. La paleta de colores y el texto del veredicto reflejan directamente los hallazgos de [12, 2] sobre prominencia visual y lenguaje concreto.

5.10. Arquitectura de Backend en la Nube

Este bloque describe el modelo serverless del backend del Filtro F1, con sus componentes de AWS Lambda y API Gateway, el efecto del cold start en el presupuesto de latencia y las capacidades de seguridad que aporta API Gateway al diseño.

5.10.1. Arquitectura Serverless para el Backend del Filtro F1

En una arquitectura serverless, el proveedor gestiona la infraestructura y asigna recursos de cómputo dinámicamente según llega cada solicitud. El backend del Filtro F1 corre sobre AWS Lambda [3], que ejecuta funciones en respuesta a eventos HTTP gestionados por Amazon API Gateway. Las funciones Lambda se activan por solicitud, escalan solas ante cambios de carga y no cuestan nada cuando no se usan, algo ideal para un backend de análisis de seguridad con tráfico variable.

Un punto crítico de rendimiento en serverless es el cold start: si una función Lambda lleva un rato sin recibir solicitudes, el proveedor libera sus recursos, y la siguiente solicitud tiene que reinicializar todo el entorno, agregando entre 200 y 1.000 ms de latencia según el runtime y el tamaño del paquete. Para el Objetivo Específico 4, que pide menos de 5 segundos de respuesta total, el cold start es el riesgo de latencia más importante del lado del servidor, y conviene mitigarlo con provisioned concurrency, que mantiene instancias ya inicializadas, o con solicitudes periódicas de calentamiento.

API Gateway, como capa entre la app móvil y las funciones Lambda, agrega capacidades útiles para la seguridad: throttling para prevenir abuso, autenticación mediante API keys o tokens JWT, y control de CORS para restringir qué orígenes pueden llamar a la API. Todo esto complementa los controles del lado del cliente (MASVS-NETWORK) y forma la capa de defensa en profundidad del servidor.

Esta arquitectura serverless define el presupuesto de latencia completo del sistema: a los menos de 100 ms de decodificación del QR se suman el tiempo de transmisión al backend, el posible cold start de Lambda y las respuestas en paralelo de las APIs de threat intelligence. Entender cada uno de estos componentes es indispensable para justificar el objetivo de latencia del Objetivo Específico 4 y para calibrar bien la duración de la pantalla de carga con skeleton o shimmer del Objetivo Específico 2, de modo que cubra el tiempo real de análisis sin dar sensación de lentitud.

CAPÍTULO 6

Antecedentes

6.1. Introducción al estado del arte

El problema de este trabajo es que un usuario no puede saber a dónde lo lleva una URL antes de que su teléfono la abra al escanear un código QR. Ese problema cruza cuatro puntos de investigación que hasta ahora han avanzado bastante por separado: (i) el estudio del código QR como vector de ataque y qué tan susceptibles son los usuarios; (ii) la detección automática de URLs maliciosas, desde listas negras hasta clasificadores que analizan cómo está escrita la URL; (iii) los sistemas de reputación y las plataformas colaborativas de *threat intelligence*; y (iv) el campo de *usable security*, que estudia cuándo una advertencia de seguridad realmente cambia lo que hace la persona que la recibe.

Al revisarlas juntas aparece un desbalance que explica por qué este proyecto tiene sentido. Los tres primeros puntos están bien desarrollados y con resultados medibles: hay taxonomías completas de ataques sobre QR [21], estudios de campo sobre susceptibilidad [39], clasificadores que reportan precisiones arriba del 95 % [23, 22] y una infraestructura de reputación de URLs funcionando a escala mundial [13, 17, 30, 1, 26]. El cuarto punto, en cambio, ha mostrado una y otra vez que detectar bien no basta para proteger: entre el 25 % y el 70 % de los usuarios ignora la advertencia y sigue adelante, según el tipo de amenaza [2].

Lo que esto implica es que hoy el *quishing* no está limitado por la falta de capacidad para detectar, sino porque esa capacidad no se aplica en el momento y en el aparato donde ocurre el ataque, y porque el resultado no se comunica de forma que el usuario pueda hacer algo con él. Este capítulo revisa los trabajos de cada punto, compara sus enfoques y resultados, señala qué aporta y qué le falta a cada uno, y delimita el vacío que el prototipo busca llenar.

Procedimiento de la revisión

Varias conclusiones de este capítulo dicen que algo no existe en la literatura. Para que esas afirmaciones se puedan verificar y quede claro hasta dónde llegan, conviene explicar cómo se hizo la búsqueda.

Se buscó en IEEE Xplore, ACM Digital Library, Scopus, SpringerLink, arXiv y Google Scholar,

además de la documentación oficial de los proveedores de reputación y de las plataformas móviles, y de informes públicos de la industria de seguridad. Los términos de búsqueda combinaron exttt"QR code.ᴬND (phishing OR quishing), extttquishing, exttt"malicious URL detection.ᴬND (mobile OR lexical), exttt"phishing blacklist.ᴬND (coverage OR "zero-hour"), exttt"security warning.ᴬND (effectiveness OR usable security) AND mobile y extttÜRL reputation.ᴬND (aggregation OR "multi-source"), en inglés y en español. El rango de años fue 2008–2025, ampliado hacia atrás solo para incluir trabajos fundacionales que siguen vigentes por la cantidad de veces que se citan.

Se incluyeron trabajos publicados en conferencias o revistas indexadas, estándares internacionales y documentación oficial de las tecnologías que se usan en el prototipo. Se dejaron fuera los trabajos sobre generación o procesamiento de imagen del símbolo QR sin componente de seguridad, las propuestas de firmar códigos QR con esquemas criptográficos que obligan a cambiar el estándar o la infraestructura de quien emite el código, y las publicaciones sin acceso al texto completo. La selección se hizo leyendo título y resumen, y después el texto completo de los que quedaron.

Esto no es una revisión sistemática formal: no hubo doble revisión independiente ni protocolo registrado de antemano. Por eso, cuando más adelante se diga que algo no se encontró, hay que leerlo con esa precisión: no se afirma que esa evidencia no exista, sino que no apareció con la búsqueda descrita aquí. Es una limitación de esta revisión y se declara como tal.

Hay un segundo punto que conviene aclarar desde el inicio. Buena parte de lo que se sabe sobre *quishing*, sobre todo en cuanto a volúmenes y crecimiento, viene de informes de empresas de seguridad [38, 18, 29] y no de publicaciones arbitradas. Esas fuentes sirven porque tienen acceso a telemetría que nadie más tiene, pero no pasan por revisión de pares y casi nunca explican su metodología con suficiente detalle como para reproducirla. Por eso en este capítulo se usan para dimensionar el fenómeno, no para sostener decisiones técnicas; esas se apoyan en publicaciones indexadas, estándares y documentación oficial.

6.2. Revisión de antecedentes relevantes

6.2.1. El código QR como vector de ataque: caracterización y evidencia empírica

El trabajo que abrió este punto es el de Krombholz et al. [21], la primera taxonomía ordenada de ataques sobre códigos QR vista desde *usable security*. Su aporte principal fue separar dos cosas: los ataques que manipulan el código físico (pegarle una calcomanía encima, reimprimir un cartel) y los que aprovechan lo que está codificado adentro. Identificaron las URLs como el contenido más peligroso y más común y, lo más importante para este trabajo, plantearon el problema como un asunto de usabilidad y no solo de criptografía o de integridad del código: el usuario no puede verificar lo que no puede leer.

La evidencia empírica clave viene de Vidas et al. [39]. En su estudio *QRishing* pegaron códigos QR en lugares públicos reales y midieron cuánta gente los escaneaba de verdad. Encontraron que los usuarios de teléfonos inteligentes caen bastante más en phishing por QR que en phishing por enlace directo, lo que confirmó con números lo que Krombholz et al. habían planteado a nivel conceptual. Metodológicamente destaca porque midieron conducta real en la calle en lugar de preguntarle a la gente qué haría, evitando así que las respuestas salieran sesgadas por querer quedar bien. También documentaron que la mayoría escaneó por pura curiosidad, sin fijarse antes en el destino.

Los dos trabajos comparten un problema de fecha: son de 2013 y 2014, o sea antes de que la pandemia de COVID-19 disparara el uso de códigos QR a partir de 2020 [35]. En el escenario que ellos describen, escanear un QR requería bajar una app aparte; hoy la cámara del teléfono lo hace sola, lo que quita todavía más fricción y hace menos probable que alguien se detenga a pensar. Sus

conclusiones sobre susceptibilidad no solo siguen siendo válidas, sino que probablemente el problema empeoró; pero las cifras concretas que reportan hay que tomarlas como piso, no como estimación actual.

Para saber cómo está el fenómeno hoy hay que recurrir a informes de la industria. Insikt Group [18] registró un aumento cercano al 248 % en menciones a *quishing* en fuentes de inteligencia entre el tercer y el cuarto trimestre de 2023 y, más importante que la cifra, documentó que el ataque se integró a plataformas de *phishing-as-a-service* como Tycoon 2FA y Greatness. Eso significa que ya no hace falta saber hacerlo: cualquiera puede comprar el servicio, lo que amplía muchísimo el número de atacantes posibles. Unit 42 de Palo Alto Networks [29] explicó el incentivo: plataformas como Microsoft 365 y Google Workspace revisan el texto de los correos pero no decodifican los códigos QR que vienen como imagen, así que el QR funciona como forma de esquivar esos filtros.

Lo valioso de estos informes no son tanto sus cifras de crecimiento, cuya metodología no se puede reproducir y cuyas bases de comparación cambian de un reporte a otro, sino cómo describen el incentivo económico y la forma de evadir los filtros. Eso sí se puede verificar por otro lado, y es lo que sostiene la premisa central de este trabajo: si el filtro de correo no puede leer el QR, el único lugar donde la URL se vuelve legible es el teléfono del usuario al momento de escanear.

6.2.2. Detección de URLs maliciosas: listas negras, heurísticas y aprendizaje automático

La referencia obligada aquí es Sahoo, Liu y Hoi [34], que ordenan las técnicas de detección en tres grupos (listas negras, heurísticas y aprendizaje automático) y explican qué gana y qué pierde cada una en cobertura, latencia, falsos positivos y capacidad de detectar amenazas nuevas. Su aporte es sobre todo organizativo y sigue siendo el marco más usado; su límite es la fecha, anterior a que se generalizaran los transformadores para clasificar texto, así que lo que dicen sobre aprendizaje profundo quedó desactualizado.

El trabajo de Sheng et al. [36] es la evaluación más citada sobre los límites de las listas negras. Midieron cuánto cubrían varias listas en operación y cuánto tardaban en incorporar sitios nuevos, y encontraron que las más grandes alcanzaban alrededor del 20 % de los sitios de phishing en el momento en que la URL se creaba, con una cobertura que mejoraba conforme el sitio llevaba más tiempo circulando y siendo denunciado. De ahí salió el concepto de *zero-hour phishing* como límite estructural de este enfoque.

Ahora bien, aplicar ese hallazgo tal cual al día de hoy sería un error, y vale la pena señalarlo porque afecta directamente cómo se justifica el filtro F1 de este proyecto. El estudio es de 2009, cuando los sistemas de reputación eran mucho menos maduros; las plataformas actuales actualizan sus bases en cuestión de minutos con análisis automatizado a gran escala. El 20 % no se puede trasladar al presente. Lo que sí sigue vigente, y es lo que sostiene el argumento, es la limitación de fondo: los informes de APWG de 2024 y 2025 [4, 5] muestran que los sitios maliciosos que duran menos de una hora antes de ser indexados siguen siendo una parte importante de los ataques, con más de un millón de ataques únicos solo en el primer trimestre de 2024. Ninguna lista puede contener algo que todavía no ha sido visto, por más rápido que se actualice. La conclusión correcta no es que las listas negras ya no sirvan, sino que son necesarias pero no suficientes: cubren muy bien y muy rápido lo que ya se conoce, y hay que complementarlas con algo que evalúe lo que no.

El tamaño de esa brecha se puede estimar con los mismos informes de APWG [4, 5], que documentan un flujo constante de dominios de phishing recién creados y una porción importante de sitios que duran menos de lo que toma indexarlos. Mientras más ataques usen dominios registrados especialmente para una campaña, menos alcanza a cubrir cualquier lista en el instante en que el usuario escanea.

Es probable que en el *quishing* esa proporción sea al menos parecida a la del phishing por enlace directo, porque repartir códigos QR físicamente favorece campañas cortas y localizadas, para las que registrar un dominio propio resulta cómodo. Hay que subrayar, eso sí, que esto es un supuesto derivado de cómo opera el ataque y no una medición: la revisión no encontró estudios que cuantifiquen específicamente cuántos dominios únicos se usan en campañas de *quishing*. Se declara como supuesto y no se usa para sostener ninguna decisión de diseño; solo indica hacia dónde apunta el efecto esperado.

Frente a este límite, el enfoque que clasifica URLs por sus características es el más prometedora para un contexto móvil. Ma et al. [23], en un trabajo de KDD '09 que abrió el camino, mostraron que con datos sacados del nombre de dominio y de cómo está escrita la URL se puede llegar a precisiones cercanas al 99 % detectando phishing conocido, sin necesidad de visitar el sitio. Le, Markopoulou y Faloutsos [22] llevaron la idea al extremo con *PhishDef*, que usa únicamente características de la URL y reporta precisiones arriba del 95 %, con la ventaja de degradarse de forma controlada cuando los datos vienen con ruido, gracias al aprendizaje en línea.

Lo que hace decisivos estos trabajos para una app móvil no es su precisión, sino algo que se desprende de no tener que abrir el sitio. De ahí salen tres ventajas. No hay que esperar una petición HTTP al destino, que además puede ser malicioso. No se expone la infraestructura del analizador ante ese destino. Y, sobre todo, no se le avisa a nadie más que un usuario específico está por entrar a una URL específica. Esto último es un tema de privacidad que la literatura revisada apenas menciona de pasada, pero que en un producto para usuarios finales pesa bastante más.

Los límites de este enfoque también importan. Los resultados salieron de conjuntos de datos históricos y en condiciones de laboratorio; ninguno de los trabajos revisados reporta cómo se comporta en uso real sobre un teléfono, ni bajo las restricciones de cómputo y batería que eso implica. Además, las características de la URL pierden efectividad ante técnicas de evasión que trabajos posteriores ya documentaron: usar acortadores, aprovechar dominios legítimos comprometidos o generar nombres por algoritmo que estadísticamente se ven igual de inocentes que los normales. Por eso este trabajo pone el análisis heurístico en un filtro posterior (F2) y no como único mecanismo.

6.2.3. Sistemas de reputación y plataformas colaborativas de threat intelligence

La infraestructura en la que se apoya el filtro F1 está documentada sobre todo en manuales técnicos oficiales y no en publicaciones arbitradas, lo cual ya dice algo sobre el estado del arte en esta área.

Google Safe Browsing [13] es el sistema de reputación con más URLs indexadas, y es el que alimenta las advertencias de Chrome, Safari y Firefox. Su Lookup API v5 permite consultar URLs una por una en tiempo real, normalmente en menos de 200 ms. Lo que aporta al diseño propuesto es llevar al escaneo móvil una cobertura que ya existe en el navegador de escritorio; su desventaja es que no se sabe con qué criterio clasifica, así que no hay forma de auditar por qué marcó o no marcó una URL.

VirusTotal [17] hace algo distinto y complementario: en vez de mantener su propia lista, junta los veredictos de más de setenta motores antivirus y escáneres de URL. La ventaja es que se reducen los falsos negativos, porque una URL que se le escapa a un motor puede ser detectada por otro. La desventaja pesa mucho en un prototipo académico: el plan gratuito permite solo cuatro consultas por minuto y quinientas al día. Ese tope limita cuánta validación empírica se puede hacer y obliga a diseñar el filtro para que siga funcionando cuando esta fuente no esté disponible.

PhishTank [30], de Cisco Talos, y URLhaus [1], de abuse.ch, funcionan de forma colaborativa: la comunidad reporta las URLs y estas se verifican por votación o curaduría. Lo que aportan es rapidez

para incorporar campañas nuevas, más que los sistemas totalmente automatizados en ciertos nichos. Su límite es que dependen de qué tan activa esté la comunidad, lo que hace que la cobertura sea despareja según la región y el idioma. Eso pega directo en el contexto guatemalteco y latinoamericano de este trabajo, donde seguramente se reporta bastante menos que en inglés. MISP [26] completa el panorama como el estándar de hecho para intercambiar indicadores de compromiso. Su relevancia aquí es indirecta pero real: varias de las fuentes integradas exportan o consumen datos en formatos compatibles con MISP, y eso explica que el ecosistema funcione junto.

Al mirar estas plataformas en conjunto aparece un vacío que ninguna documentación oficial cubre: en la literatura revisada no hay un estudio independiente que compare qué tanto coinciden estas fuentes sobre un mismo conjunto de URLs, ni que describa cómo se comporta un veredicto combinado frente a los veredictos por separado. Cada proveedor publica lo que él hace; nadie publica cómo funcionan juntos. La validación del filtro F1 prevista en este trabajo, aunque sea de alcance modesto, aporta algo en esa dirección.

6.2.4. Soluciones comerciales de escaneo seguro de QR

En las tiendas de apps ya hay soluciones que atacan parte del problema. Herramientas como *Safe QR Scan – Secure QR Scanner* y *SafeQR Reader · Block Scams*, disponibles en la App Store, revisan la URL antes de redirigir, comparándola contra bases de datos de dominios maliciosos conocidos.

Su mérito es haber demostrado en el mercado que el punto de intervención funciona: confirman que meterse entre el escaneo y la apertura del navegador es viable técnicamente y que a los usuarios no les molesta. Pero tienen tres limitaciones de fondo. Primero, como ya documentó la literatura revisada [36, 39], trabajan casi todas con listas negras, que no alcanzan contra dominios recién creados, justamente los que dominan en campañas de *quishing*. Segundo, son productos cerrados: no publican su arquitectura, ni sus fuentes de datos, ni sus tasas de error, así que no hay manera de evaluarlas de forma independiente ni de compararlas científicamente. Tercero, ninguna explica en qué basó sus decisiones de interfaz, aun cuando la literatura de *usable security* dice que ahí es donde fallan la mayoría de sistemas de advertencia.

Conviene aclarar que este trabajo no incluye una comparación formal contra estas apps comerciales; esa exclusión está declarada en el capítulo de alcance. Se revisan aquí para mostrar cómo está la práctica actual y para dejar claro que lo que se propone –una arquitectura documentada, con fuentes declaradas y decisiones de interfaz fundamentadas– juega en otro plano que el de competir en funciones con productos de mercado.

6.2.5. Usable security y efectividad de advertencias de seguridad

Este punto aporta el fundamento de las decisiones de interfaz. Puede sostenerse, además, que es la que explica por qué la sola disponibilidad de detección eficaz no ha resuelto el problema.

Cranor [11] propuso el marco del humano en el ciclo, que descompone el comportamiento seguro en tres condiciones encadenadas: la persona tiene que notar la señal, entender qué significa y actuar en consecuencia. Sirve como diagnóstico, porque permite ubicar en cuál de los tres eslabones está fallando un sistema concreto en lugar de echarle la culpa genéricamente al descuido del usuario. Su límite es que es un modelo conceptual y no un instrumento de medición, así que validarlo directamente es complicado.

Egelman, Cranor y Hong [12] aportaron evidencia experimental sobre qué hace efectiva una advertencia de phishing en el navegador: interrumpir lo que la persona está haciendo, destacar visualmente, usar lenguaje concreto en vez de técnico y aparecer en el momento en que más importa. Akhawe y Felt [2] llevaron el análisis a datos reales de Chrome, con un hallazgo muy relevante para

este proyecto: la tasa de continuación, o sea cuánta gente ignora la advertencia y sigue de todos modos, fue de alrededor del 70 % en advertencias de SSL frente a cerca del 25 % en advertencias de malware. Esa diferencia muestra que la reacción depende de qué tan grave se percibe la amenaza y de qué tan bien se entiende, no de que la gente ignore los avisos por costumbre. La implicación de diseño es directa: distinguir claramente entre niveles de veredicto no es un detalle estético, es un requisito para que la advertencia funcione.

El problema común de este punto es el contexto: los tres trabajos estudian advertencias en navegadores de escritorio. Nadie ha validado si sus hallazgos se trasladan al entorno móvil, que es distinto: pantalla chica, interacción táctil, uso frecuente en movimiento y con prisa, y sin barra de direcciones para verificar. Es, entonces, una extrapolación razonable pero no comprobada, y este trabajo la asume explícitamente como tal.

6.2.6. Estándares y documentación técnica de referencia

El estándar ISO/IEC 18004 [19, 20] define cómo funciona el código QR: modos de codificación, niveles de corrección de errores y versiones de tamaño. Importa como antecedente por dos razones. Por un lado, delimita técnicamente qué puede llevar adentro un QR y, por lo tanto, qué tiene que clasificar el módulo de escaneo antes de decidir si corresponde analizarlo. Por otro, y esto pesa más para el planteamiento del problema, el estándar no incluye ningún mecanismo para autenticar quién generó el código ni para verificar que el contenido no fue alterado. No es un defecto de la norma, porque se diseñó en 1994 para trazabilidad industrial [19] y no como canal de confianza, pero ahí está la raíz técnica de la vulnerabilidad: dentro del estándar no hay forma de distinguir un QR legítimo de uno pegado encima. Cualquier solución tiene que ubicarse por encima de la capa del código, lo que respalda la decisión de este trabajo de analizar la URL en texto (RFC 3986) y no el símbolo QR.

El OWASP MASVS v2.1 [28] da el marco para verificar la seguridad de la app misma, con categorías que aplican directo al prototipo: MASVS-STORAGE para el historial local cifrado, MASVS-NETWORK para la comunicación con el backend y las APIs, y MASVS-PLATFORM para el manejo de permisos de cámara y ubicación. Lo útil es que ofrece criterios verificables en lugar de buenas prácticas sueltas; su límite reconocido es que es un estándar de requisitos y no una guía de implementación. La documentación oficial de cada plataforma (App Transport Security [7] y Apple Platform Security [10] en iOS, Network Security Configuration [14] en Android) complementa el estándar con los mecanismos concretos.

Por último, la documentación de las librerías y APIs nativas de escaneo (VNDetectBarcodesRequest del framework Vision [9], ML Kit Barcode Scanning [16] y react-native-vision-camera [33]) confirma que se puede procesar todo en el teléfono en menos de 100 ms. Este dato es el que hace posible el objetivo de mantener la latencia total por debajo de cinco segundos: si hubiera que mandar la imagen a un servidor para decodificarla, ni el tiempo ni la privacidad serían alcanzables.

6.3. Comparación de los trabajos revisados

Poniendo los antecedentes uno junto a otro, se pueden ordenar según en qué momento del ataque intervienen y qué tipo de evidencia aportan.

extbfPor enfoque metodológico se distinguen cuatro grupos con características distintas. Los estudios de campo que miden conducta real [39] son los más cercanos a la realidad, pero con muestras pequeñas y difíciles de repetir. Las evaluaciones sobre conjuntos de datos [36, 23, 22] dan métricas precisas y comparables, aunque en condiciones de laboratorio que no reflejan el uso real. Los estudios a gran escala con datos de producción [2] combinan volumen y realismo, pero solo los puede hacer

quien controla la plataforma. Y los informes de la industria [18, 29] ofrecen la foto más actual del panorama de amenazas, a cambio de no pasar por revisión de pares ni ser reproducibles.

extbfPor punto de intervención la comparación dice más. Los sistemas de reputación [13, 17, 30, 1] actúan sobre la URL una vez que el ecosistema ya la conoce; cubren mucho y responden rápido, pero no ven lo nuevo. Los clasificadores léxicos [23, 22] actúan sin importar el historial de la URL, así que pueden evaluar dominios recién registrados, a cambio de más falsos positivos. Los filtros de correo actúan antes de que el mensaje llegue, pero como documenta Unit 42 [29] no leen el QR y quedan fuera de juego. Las advertencias del navegador [12, 2] actúan después de navegar, cuando la petición al servidor del atacante ya salió. Y las apps comerciales de escaneo seguro actúan en el punto correcto, entre el escaneo y la apertura, pero con una detección insuficiente.

De esta comparación sale el hallazgo central del capítulo: extbfel punto donde la intervención es posible coincide con el punto donde ningún trabajo revisado aplica al mismo tiempo detección multicapa y una comunicación bien fundamentada del veredicto. El momento entre leer el código y la redirección es el único en que la URL se puede ver, el ataque todavía no se consuma y el usuario está en posición de decidir. Los sistemas de reputación saben detectar pero no operan ahí; las apps comerciales operan ahí pero sin detección suficiente ni criterio para comunicar; y la literatura de *usable security* sabe cómo comunicar pero no ha estudiado este contexto.

extbfPor cómo tratan la comunicación del veredicto el desbalance es todavía más marcado. De todos los antecedentes revisados, solo los de *usable security* [11, 12, 2] se ocupan explícitamente de cómo presentar un resultado de seguridad, y ninguno lo hace en el contexto móvil de escaneo de QR. Los trabajos técnicos de detección [34, 36, 23, 22] terminan en la métrica de clasificación y no entran en la interfaz. Los proveedores de reputación [13, 17, 30, 1] entregan un veredicto estructurado y dejan toda la presentación en manos de quien lo integre. Esa separación entre quien detecta y quien comunica es justamente donde se ubica el aporte de este trabajo.

La siguiente tabla resume la comparación de los principales antecedentes según su enfoque, lo que aportan y lo que les falta para los fines de este proyecto.

Tabla 6.1: Comparación de los trabajos revisados según enfoque, aporte y limitación.

Trabajo / fuente	**Enfoque metodológico**	**Aporte principal**	**Limitación relevante**
Krombholz et al. [21]	Taxonomía conceptual	Formula el problema como cuestión de usabilidad; identifica la URL como vector dominante	Anterior al escaneo nativo y a la adopción masiva post-2020
Vidas et al. [39]	Estudio de campo, conducta observada	Evidencia cuantitativa de mayor susceptibilidad al QR frente a URL directa	Muestra reducida; magnitudes no extrapolables al ecosistema actual
Sahoo et al. [34]	Revisión sistemática	Taxonomía de técnicas que fundamenta la arquitectura multicapa F1–F4	Tratamiento desactualizado del aprendizaje profundo
Sheng et al. [36]	Evaluación empírica de sistemas	Establece el *zero-hour phishing* como límite estructural de las listas	Cifra del 20 % no transferible a plataformas actuales
Ma et al. [23]	Clasificación sobre conjunto de datos	Precisión cercana al 99 % sin visitar el sitio destino	Condiciones de laboratorio; sin despliegue móvil

(continúa en la página siguiente)

(continuación de la Tabla 6.1)

Trabajo / fuente	Enfoque metodológico	Aporte principal	Limitación relevante
Le et al. [22]	Clasificación léxica en línea	Precisión superior al 95 % solo con características de la URL	Vulnerable a acortadores y a dominios comprometidos
Google Safe Browsing [13]	Documentación técnica oficial	Mayor cobertura indexada; latencia inferior a 200 ms	Criterio de clasificación opaco, no auditable
VirusTotal [17]	Documentación técnica oficial	Agregación de más de 70 motores; reduce falsos negativos	Plan gratuito: 4 consultas/min y 500/día
PhishTank [30] / URLhaus [1]	Plataforma colaborativa	Rapidez ante campañas emergentes	Cobertura desigual por región e idioma
Cranor [11]	Modelo conceptual	Marco diagnóstico de los tres eslabones del comportamiento seguro	No operacionalizado como instrumento de medición
Egelman et al. [12]	Experimento controlado	Criterios de diseño de advertencias basados en evidencia	Contexto exclusivamente de navegador de escritorio
Akhawe y Felt [2]	Estudio de campo a gran escala	La severidad percibida modula la respuesta: tasa de continuación del 70 % en SSL frente al 25 % en malware	No validado en contexto móvil
ISO/IEC 18004 [19, 20]	Estándar internacional	Delimita el contenido codificable y la lógica de clasificación	No define autenticación del emisor ni integridad del contenido
OWASP MASVS [28]	Estándar de verificación	Criterios verificables de seguridad de la aplicación	Estándar de requisitos, no metodología de implementación
Safe QR Scan, *SafeQR Reader* (App Store)	Producto de mercado; sin documentación técnica pública	Validan la viabilidad y la aceptación del punto de intervención	Cerradas e inauditables; detección basada en lista negra

6.4. Aportes y limitaciones de los trabajos revisados

6.4.1. Aportes principales

Los antecedentes revisados le dan a este proyecto cuatro bases distintas. Del bloque que caracteriza el vector [21, 39] se toma la confirmación de que el problema es de usabilidad y no solo técnico, junto con la evidencia de que caer en *quishing* es algo estructural y no un descuido individual. Del bloque de detección [34, 36, 23, 22] se toma la clasificación de técnicas que sustenta la arquitectura multicapa F1–F4, y la evidencia de que ninguna técnica sola alcanza. Del bloque de infraestructura [13, 17, 30, 1, 26] se toma que existen fuentes de reputación consultables en tiempo real con latencias compatibles con el objetivo de cinco segundos, además del modelo colaborativo en que se apoya la

función de reporte del prototipo. Del bloque de *usable security* [11, 12, 2] se toman los criterios de diseño de advertencias que sustentan mostrar el veredicto en dos niveles de detalle. Y los estándares [28, 19, 20] junto con la documentación técnica [9, 16, 33, 10, 7, 14] aportan los criterios verificables de cumplimiento y la confirmación de que procesar todo en el teléfono es viable.

6.4.2. Limitaciones identificadas

Mirando críticamente esos mismos trabajos aparecen seis limitaciones que, juntas, delimitan el vacío de investigación.

Desactualización de la evidencia empírica sobre el vector. Los estudios de referencia sobre susceptibilidad [21, 39] son anteriores a que el escaneo de QR se integrara a la cámara del teléfono y a la adopción masiva posterior a 2020. Lo que concluyen sigue siendo válido en lo cualitativo, pero sus números ya no se pueden trasladar al escenario actual.

Transferencia no validada de la evidencia sobre advertencias. Los hallazgos sobre qué tan efectivas son las advertencias [12, 2] vienen todos de navegadores de escritorio. Aplicarlos al escaneo móvil (pantalla chica, uso en movimiento, sin barra de direcciones) es una extrapolación razonable que ninguna publicación revisada ha comprobado.

Brecha entre desempeño de laboratorio y despliegue real. Los clasificadores léxicos reportan precisiones del 95 % al 99 % [23, 22] sobre conjuntos de datos históricos, sin evidencia de cómo se comportan bajo las restricciones de cómputo, batería y conectividad de un teléfono en uso real.

Ausencia de evaluación independiente de las fuentes de reputación. No se encontró literatura arbitrada que compare qué tanto coinciden Google Safe Browsing, PhishTank y VirusTotal sobre un mismo conjunto de URLs, ni que describa cómo se comporta un veredicto combinado. Lo único disponible es la documentación de cada proveedor.

Opacidad de las soluciones comerciales. Las apps existentes no publican su arquitectura, ni sus fuentes, ni sus tasas de error, así que no se pueden auditar. Eso impide compararlas científicamente y también aprender de las decisiones que tomaron.

Fragmentación entre detección y comunicación. Ningún trabajo revisado junta en una misma propuesta la detección multicapa y un diseño fundamentado de cómo comunicar el veredicto. La literatura técnica termina en la métrica; la de usabilidad arranca cuando la advertencia ya está formulada. El espacio de en medio, o sea convertir un veredicto técnico en una decisión que un usuario común pueda tomar, en el teléfono y en menos de cinco segundos, sigue sin abordarse de forma integrada.

A estas se suma una limitación de contexto: extbfno hay producción académica sobre el fenómeno en el entorno latinoamericano y guatemalteco. Toda la literatura revisada viene de contextos anglosajones y europeos, mientras que las plataformas colaborativas de reporte seguramente cubren menos las campañas en español dirigidas a la región. Eso sugiere que el límite de las listas negras podría ser incluso más grave aquí que donde se midió.

6.5. Fundamentación del proyecto propuesto

Los antecedentes sustentan el proyecto en cuatro planos, y conviene dejar clara la correspondencia entre cada limitación identificada y la decisión de diseño que la responde.

Fundamentación del punto de intervención. Cuatro puntos de evidencia apuntan a lo mismo. Krombholz et al. [21] muestran que no se puede inspeccionar el contenido antes de escanear. Vidas et al. [39] miden qué tan susceptible resulta la gente por eso. Unit 42 [29] documenta que los filtros de correo no ven el QR. Y la falta de mecanismos de autenticación en ISO/IEC 18004 [19, 20] impide resolverlo en la capa del código. Todo junto indica que el único punto de la cadena del ataque que sigue sin protección automatizada es el teléfono del usuario en el instante del escaneo. El objetivo general del trabajo, meterse entre la lectura del código y la redirección, sale directamente de esa evidencia y no de una preferencia de diseño.

Fundamentación de la arquitectura multicapa. La clasificación de Sahoo et al. [34] y la evidencia sobre los límites de cobertura de las listas [36], puesta en contexto con los datos de APWG de 2024 y 2025 [4, 5], sustentan que ninguna técnica sola resuelve el problema. De ahí la arquitectura F1–F4. Que F1 sea el primer filtro se apoya en lo que ya se sabe de las fuentes de reputación [13, 17, 30]: responden rápido, aciertan mucho sobre amenazas conocidas y cuestan poco en cómputo, lo que las vuelve el punto de decisión inicial más lógico bajo el límite de cinco segundos. Y saber que no alcanzan es, al mismo tiempo, la razón de que haya filtros después.

Fundamentación de las decisiones de interfaz. El marco de Cranor [11] y los resultados de Egelman et al. [12] y de Akhawe y Felt [2] sustentan mostrar el veredicto en dos niveles de detalle con divulgación progresiva, usar lenguaje concreto en vez de técnico, diferenciar visualmente los niveles de riesgo e interrumpir el flujo a propósito. Que la tasa de continuación varíe entre el 25 % y el 70 % según el tipo de amenaza [2] sustenta específicamente que los tres veredictos (seguro, sospechoso, malicioso) tengan que distinguirse sin ambigüedad y no ser matices de una misma escala.

Fundamentación de las decisiones técnicas. La documentación de las APIs nativas de visión [9, 16] y de react-native-vision-camera [33] confirma que procesar en el teléfono con menos de 100 ms es viable, condición necesaria para el presupuesto de tiempo total. El MASVS [28] sustenta los controles de almacenamiento cifrado, seguridad de red y permisos mínimos, bajo un criterio verificable: una app que existe para proteger al usuario tiene que ser ella misma auditablemente segura.

Aporte diferencial. Con todo esto sobre la mesa, el aporte de este trabajo se puede definir con precisión y sin inflarlo. No consiste en proponer una técnica nueva de detección, porque el proyecto integra fuentes que ya existen, ni en superar a las apps comerciales, comparación que quedó fuera del alcance. Consiste en tres cosas: (i) integrar de forma documentada y auditable, en el punto correcto, la detección multicapa y una comunicación fundamentada del veredicto, lo que responde a la sexta limitación; (ii) aportar evidencia empírica sobre cómo se comporta un veredicto combinado de tres fuentes de reputación, lo que responde a la cuarta; y (iii) aplicar los principios de *usable security* al escaneo móvil de QR, documentando la extrapolación que la segunda limitación deja pendiente. Las restricciones también son explícitas: solo F1 se implementa de extremo a extremo, la validación de usabilidad no alcanza representatividad estadística y no se compara con productos comerciales.

6.6. Conclusión: vacíos identificados y necesidad del proyecto

Revisar críticamente los antecedentes lleva a una conclusión: hoy el *quishing* no está limitado por falta de capacidad técnica para detectar. Esa capacidad existe, funciona a escala mundial y se puede consultar por interfaces públicas en milisegundos. Lo que falta es aplicarla en el lugar y en el momento donde el ataque se concreta, y cerrar la distancia entre quienes desarrollan la detección y quienes estudian cómo explicarle el resultado a un usuario que no es experto.

Los vacíos identificados son cuatro. Hay un extbfvacío de aplicación: la infraestructura de reputación está desplegada en navegadores de escritorio y en filtros de correo, pero no en el escaneo móvil, que es justamente el único punto donde la URL de un QR se vuelve legible antes de navegar. Hay un extbfvacío de integración: la literatura técnica de detección y la de *usable security* avanzaron sin cruzarse, y por eso ninguna propuesta revisada aborda de forma unificada la detección multicapa junto con el diseño de cómo comunicar el veredicto. Hay un extbfvacío de evaluación: no hay evidencia arbitrada sobre qué tanto coinciden las principales fuentes de reputación ni sobre cómo se comporta un veredicto combinado, ni tampoco validación de los hallazgos sobre advertencias en contexto móvil. Y hay un extbfvacío de transparencia: las únicas soluciones que operan en el punto correcto son productos cerrados, imposibles de auditar y con un mecanismo insuficiente, lo que impide que el conocimiento sobre este problema se acumule.

Estos vacíos justifican el proyecto. Desarrollar un prototipo de app móvil que se meta antes de la redirección al momento de escanear, que combine en tiempo real los veredictos de tres fuentes independientes de reputación con un límite de cinco segundos, y que muestre el resultado en una interfaz diseñada según los principios documentados de *usable security*, responde directamente a cada uno de ellos. Documentar de forma abierta su arquitectura, sus fuentes y sus resultados de validación, incluidas las limitaciones, atiende además el vacío de transparencia y deja la puerta abierta para que otros trabajos extiendan la propuesta hacia los filtros heurístico, de *sandbox* y de aislamiento de navegación que ya están previstos en el diseño.

El contexto refuerza la pertinencia del esfuerzo. Toda la literatura revisada viene de contextos anglosajones y europeos, mientras que la evidencia disponible sugiere que en Latinoamérica los códigos QR se adoptaron más rápido de lo que crecieron las prácticas de seguridad, y que las plataformas colaborativas de reporte cubren menos las campañas dirigidas a la región. Un trabajo que documente el problema y proponga una solución evaluable desde Guatemala no solo llena un vacío técnico, sino que deja un antecedente en un área donde la producción académica local todavía es escasa.

CAPÍTULO 7

Alcance

7.1. Definición del alcance

El presente protocolo se enfoca en el diseño e implementación del frontend móvil del sistema de análisis automatizado de URLs para detección de quishing, integrado dentro de un sistema compuesto por una interfaz en React Native y un motor backend desplegado en AWS. El frontend cubre el flujo completo de la experiencia de usuario e integra el Filtro F1 como primer filtro de análisis funcional end-to-end:

- **F1 — Reputación:** consumo en tiempo real del veredicto consolidado de Google Safe Browsing, PhishTank y VirusTotal; primer filtro funcional integrado de extremo a extremo.
- **Pantallas de resultado para F2.1** (análisis heurístico), **F2.2** (sandbox) y **F2.3** (vista previa visual segura): la interfaz presenta el desglose y los indicadores recibidos del backend; la implementación del motor de análisis de estos filtros forma parte del módulo backend.
- **Funcionalidades secundarias:** historial local offline (últimos 20 escaneos), reporte de QR malicioso al backend y Story Card compartible con geocodificación inversa.

El producto final es demostrable en dispositivos físicos iOS y Android; no contempla publicación en tiendas de aplicaciones.

7.2. Delimitación Funcional y Técnica

7.2.1. Funcionalidades incluidas

Tabla 7.1: Delimitación Funcional y Técnica

Módulo	Componente
Frontend	Decodificación QR nativa en tiempo real (Apple Vision / ML Kit)
Frontend	Clasificación de contenido: URL vs. no-URL
Frontend	Pantalla de carga (skeleton/shimmer) y resultado simple/extendido
Frontend	Historial local cifrado (últimos 20 escaneos, consultable offline)
Frontend	Reporte de QR malicioso al backend
Frontend	Story Card compartible con ubicación geocodificada
Frontend	Integración del Filtro F1 (consumo del veredicto de reputación vía backend)

El motor de análisis backend (filtros F2.1, F2.2 y F2.3) forma parte del sistema integral, pero su implementación queda fuera del alcance principal del presente protocolo; el frontend consume sus resultados mediante el contrato de interfaz acordado.

7.2.2. Stack tecnológico fijo

Tabla 7.2: Stack tecnológico fijo

Capa	Tecnología
Framework móvil	React Native 0.76+ (New Architecture)
Cámara y escaneo	react-native-vision-camera v4
Almacenamiento local	react-native-mmkv
Navegación	React Navigation
Geocodificación inversa	Nominatim / Google Maps API
Reputación (F1)	Google Safe Browsing v5, PhishTank, VirusTotal v3

La unidad de análisis es exclusivamente la URL textual extraída del QR (RFC 3986). No se analiza el símbolo QR en sí ni ningún otro tipo de contenido codificado.

7.3. Restricciones

- **Temporal:** diecisiete semanas de desarrollo organizadas en seis fases, contemplando un margen adicional dentro del semestre académico para atender posibles ajustes o imprevistos.
- **Equipo:** dos integrantes, cada una responsable de un módulo. Sin recursos adicionales.
- **Económica:** el sistema opera dentro de los planes gratuitos de todas las APIs externas y del AWS Free Tier. No se contempla inversión en planes de pago.
- **Hardware:** la compilación para iOS requiere acceso periódico al laboratorio de macOS de la universidad.
- **Conectividad:** el análisis requiere conexión a internet. Sin conexión, el sistema no produce veredicto y lo comunica al usuario; no se implementa análisis offline.

7.4. Exclusiones explícitas

- **Funcionales:** cuentas de usuario y autenticación; historial ilimitado; GIF animado del comportamiento del sitio (F2.3); notificaciones push; escaneo en segundo plano.
- **De contenido:** análisis de seguridad sobre datos WiFi, vCard o esquemas de pago codificados en QR; URLs con esquemas distintos de `http://` y `https://`.
- **Técnicas:** implementación del motor de análisis backend (filtros F2.1, F2.2 y F2.3); modelos de aprendizaje automático propios; análisis de HTML o JavaScript en el dispositivo; despliegue en tiendas de aplicaciones; pruebas de carga a escala.
- **De investigación:** estudios de usabilidad con muestras estadísticamente representativas; estudios de campo en entornos reales; análisis comparativo con soluciones comerciales.

CAPÍTULO 8

Metodología

8.1. Enfoque y tipo de investigación

El trabajo corresponde a una investigación aplicada de desarrollo tecnológico con validación experimental en dispositivo. No se busca generar conocimiento teórico nuevo sobre detección de URLs maliciosas, materia ya tratada en los antecedentes, sino construir un artefacto funcional que lleve capacidades de análisis existentes al único punto de la cadena del ataque donde todavía se pueden aplicar: el intervalo entre la lectura del código QR y la apertura del navegador. El método, por lo tanto, se organiza alrededor de dos preguntas verificables: si el artefacto cumple los tiempos y las tasas de acierto que lo vuelven usable, y si el veredicto que produce se entiende lo suficiente como para que el usuario decida bien.

La unidad de análisis es exclusivamente la URL textual extraída del código, según el RFC 3986, tal como se delimitó en el capítulo de alcance. No se analiza el símbolo QR como imagen ni ningún otro contenido codificable.

Conviene fijar desde el inicio una distinción que ordena todo el diseño de la evaluación. El presente trabajo mide el desempeño del módulo de frontend: la latencia del escaneo, la tasa de detección de códigos, la corrección de la clasificación de contenido, la integridad del flujo y la comprensión del veredicto por parte del usuario. No mide la capacidad de detección del motor de análisis, que depende de las fuentes de reputación y de los filtros implementados en el módulo de backend. Son dos conjuntos de criterios distintos y ninguno sustituye al otro: una pantalla muy clara que comunique un veredicto poco confiable no protege a nadie, y un veredicto acertado pero mal explicado tampoco. Cuando una métrica dependa del backend, se declara explícitamente en la sección correspondiente.

8.2. Arquitectura de la solución

El prototipo se organiza en cuatro capas, cada una con una responsabilidad que no se solapa con las demás. Esta separación no es ornamental: es la que permite que la lógica de negocio se pruebe sin cámara, sin red y sin dispositivo, lo que a su vez determina qué parte de la validación puede

automatizarse y qué parte exige hardware.

Tabla 8.1: Capas de la aplicación y su responsabilidad

Capa	Responsabilidad
Rutas	Doce pantallas con enrutamiento basado en archivos: escaneo, carga, resultado, detalle, contenido no navegable, historial, reporte, Story Card y dos pasos de onboarding
Sistema de diseño	Diecisiete componentes y los tokens de color, tipografía y espaciado, extraídos del prototipo visual de alta fidelidad
Servicios	Lógica de negocio pura y aislada: análisis, cliente HTTP, clasificación de QR, almacenamiento, ubicación, reporte y formato
Estado	Un único contexto de React que sostiene el resultado vigente, el historial y la preferencia de ubicación

La dirección de las dependencias merece una nota, porque es contraria a lo que suele esperarse. El tipo que enumera los cuatro veredictos posibles —seguro, sospechoso, malicioso e incierto— reside en el sistema de diseño, dado que cada veredicto lleva asociados color, etiqueta y tratamiento visual. En consecuencia, la capa de servicios depende del sistema de diseño y no al revés. La implicación práctica es que un quinto veredicto se agregaría en el sistema de diseño y desde ahí se propagaría al resto.

8.2.1. Plataforma y versiones

El prototipo se construye sobre el stack que recoge el cuadro 8.2. Se consignan las versiones exactas porque varias decisiones del método, en particular la que se discute en la sección siguiente, se explican por restricciones de compatibilidad entre versiones concretas y no por preferencias generales.

Tabla 8.2: Versiones verificadas del entorno de desarrollo

Componente	Versión
Expo SDK	57.0.12
React Native	0.86.2 (New Architecture)
React	19.2.3
Enrutamiento por archivos	Expo Router 57.0.12
Cámara y escaneo	expo-camera 57.0.4
Almacenamiento clave-valor	expo-sqlite 57.0.2
Lenguaje	TypeScript 6.0.3, modo `strict`
Pruebas	Jest 30.5.1 con preset `jest-expo`
Entorno de compilación	Node 20.20.2, JDK 17, Gradle 9.3.1, API mínima 24

8.2.2. Decisión sobre las librerías de escaneo y almacenamiento

El anteproyecto especificaba `react-native-vision-camera` para el escaneo y MMKV para el almacenamiento local. La implementación emplea los módulos nativos del propio SDK: `expo-camera` y el almacén clave-valor de `expo-sqlite`. La desviación se documenta aquí con su justificación técnica y con el criterio que la revertiría, porque un cambio de esta naturaleza debe quedar declarado como decisión y no descubrirse al leer el código.

El motivo es un conflicto de dependencias nativas verificado durante la fase de integración. El enrutador por archivos arrastra la versión 0.12.x de la librería de *worklets*, mientras que el núcleo de módulos del SDK en su versión 57.0.10 solo admite versiones iguales o anteriores a la 0.10.x. La combinación rompe la compilación nativa de Android con un error de símbolo ausente en tiempo de enlace, lo que obligó a fijar versiones explícitas de resolución en el manifiesto de dependencias. Adoptar los módulos del propio SDK reduce esa superficie de fallo, pues sus versiones se publican alineadas con el SDK, sin sacrificar ninguna de las capacidades que el escaneo requiere: decodificación nativa en tiempo real restringida al formato QR, control de linterna y gestión de permisos de cámara.

En el caso del almacenamiento, la razón es funcional además de práctica. El almacén clave-valor de `expo-sqlite` ofrece una interfaz *síncrona* de lectura. Esto permite resolver dos decisiones durante el primer renderizado: a qué pantalla dirigir al usuario según si ya completó el onboarding, y con qué contenido pintar el historial. Con una interfaz asíncrona, ambas decisiones ocurrirían después del primer renderizado y producirían un parpadeo visible hacia la pantalla equivocada.

El criterio de reversión está fijado de antemano y es el resultado de la prueba 1.4, descrita en la sección 8.5. Si la tasa de detección resulta inferior al 95 % o el percentil 90 del tiempo de escaneo supera los 5 s, se migra a `react-native-vision-camera` conforme al anteproyecto original, con un esfuerzo estimado de uno a dos días. Antes de concluir que la migración es necesaria se descartan tres hipótesis de menor costo: que el fallo se concentre en ángulos de 45°, caso en que correspondería ampliar el área de escaneo; que se concentre en penumbra, caso en que la linterna podría resolverlo y la interfaz debería sugerirla; o que se concentre en códigos pequeños a distancia, que sería una limitación de enfoque y no de decodificación. Si el fallo se concentra en un solo eje se documenta como limitación conocida; si se reparte en los tres, la migración queda justificada.

8.3. Procedimiento de desarrollo

El desarrollo se organiza en ocho bloques ordenados por dependencia técnica y riesgo, no por la secuencia de fases del anteproyecto. El cambio de criterio responde a una situación concreta: al iniciar la integración existían tareas de fases avanzadas terminadas y tareas de fases iniciales todavía abiertas, de modo que el orden original había dejado de describir el estado real del proyecto. La reorganización se rige por tres reglas declaradas: primero lo que desbloquea el trabajo de otras personas, después lo que elimina la mayor cantidad de riesgo técnico y al final lo que depende únicamente de tiempo disponible.

Tabla 8.3: Bloques de desarrollo y criterio de cierre

N.º	Objetivo	Criterio de cierre
0	Desbloquear el módulo de backend	Contrato de interfaz entregado
1	Escaneo real en dispositivo	Prueba 1.4 superada: punto de decisión sobre la librería
2	Persistencia local	Historial y preferencias sobreviven al reinicio
3	Integración con el backend	Filtro F1 funcional de extremo a extremo con degradación controlada
4	Completar funcionalidades parciales	Reporte, Story Card e historial operativos
5	Compatibilidad con iOS	Fuera de alcance por falta de hardware
6	Pruebas y métricas	Cobertura, mediciones en hardware y pruebas de usabilidad
7	Cierre y documentación	Informe y defensa

El bloque 1 concentra el mayor riesgo técnico del módulo, razón por la cual se adelanta: si el escaneo no cumple los tiempos, todo lo construido encima queda condicionado. El bloque 5 se mantiene en el plan aunque esté fuera de alcance, para que la exclusión sea una decisión visible y no una omisión.

8.3.1. El contrato de interfaz como instrumento

La coordinación con el módulo de backend se formalizó mediante un contrato de interfaz escrito antes de que existiera el endpoint, y conviene señalar el procedimiento con que se redactó porque tiene valor metodológico. El contrato no se derivó de lo que el backend podría ofrecer, sino del consumo real del frontend hacia atrás: las doce pantallas ya implementadas sobre datos de prueba determinaban exactamente qué campos se necesitaban, con qué tipos y en qué momento, lo que convierte a las definiciones de tipos del frontend en una especificación ya validada contra una interfaz funcional, y no en una lista de deseos.

El contrato declara formato JSON en UTF-8, marcas temporales en ISO 8601 e identificación anónima de dispositivo sin cuentas de usuario, y traslada la regla de seguridad fundamental del producto a una restricción sobre la API: ningún endpoint puede devolver HTML para que el cliente lo renderice, ni redirigir el dispositivo hacia el sitio analizado. A la fecha de corte de este informe permanecen ocho puntos del contrato sin acordar, lo que se consigna como dependencia externa y no como tarea pendiente del módulo.

8.4. Estrategia de validación

La validación se organiza en tres niveles, separados por el tipo de pregunta que cada uno puede responder. La separación es deliberada: asignar a un nivel una pregunta que no puede contestar es la forma más común de producir evidencia que parece sólida sin serlo.

8.4.1. Nivel 1: pruebas unitarias de la lógica de negocio

El primer nivel cubre la capa de servicios mediante pruebas unitarias automatizadas. A la fecha de corte el conjunto comprende 249 pruebas distribuidas en 19 suites, con un umbral de cobertura del 70 % en sentencias, ramas, funciones y líneas que interrumpe la compilación si no se alcanza.

La cobertura se mide únicamente sobre la capa de servicios. Las pantallas y el sistema de diseño quedan excluidos a propósito, no por descuido: lo que se verifica en ellos —que un color comunique riesgo, que una jerarquía de botones induzca a la decisión correcta— no es una afirmación que una prueba unitaria pueda sostener, y se valida en dispositivo y en las pruebas de usabilidad.

El aislamiento se consigue con cuatro dobles de prueba y ninguno más: sustitución global del almacén clave-valor por una implementación en memoria con la misma interfaz síncrona, reemplazo de la función de red del entorno por una simulada, sustitución del cliente HTTP en las pruebas de los servicios que lo consumen y sustitución del módulo de ubicación. En ningún momento se realizan peticiones de red reales, de modo que el conjunto es determinista y reproducible sin conectividad ni credenciales.

Dos pruebas del conjunto merecen mención porque validan el método y no solo el código. La primera fija como aserción el valor del tiempo límite de la petición de análisis: sin ella, elevarlo a 60 s dejaría pasar todas las demás pruebas sin que nadie lo advirtiera, y un parámetro con consecuencias sobre la experiencia de uso quedaría sin vigilancia. La segunda recorre la tabla de códigos QR de ejemplo de la documentación y verifica que cada entrada produzca la clasificación que la tabla declara, con lo que la documentación deja de poder divergir silenciosamente del comportamiento real.

8.4.2. Nivel 2: pruebas en dispositivo físico

El segundo nivel cubre todo lo que resulta inverificable desde una computadora: diálogos del sistema operativo, cámara, GPS real, galería, compartición y el backend en producción. Se compone de dos protocolos numerados que se detallan en la sección 8.5.

8.4.3. Nivel 3: pruebas de usabilidad

El tercer nivel evalúa lo que ningún test automatizado puede responder: si el veredicto se entiende. Se realizará con cinco participantes y medirá tres indicadores: comprensión del veredicto, con criterio de al menos el 90 %; finalización del flujo completo sin asistencia, con criterio de al menos el 85 %; y tiempo de aprendizaje, con criterio máximo de 60 s. El fundamento de medir la comprensión por separado del desempeño técnico proviene de la literatura de *usable security*: el modelo del humano en el ciclo descompone el comportamiento seguro en notar la señal, entenderla y actuar en consecuencia [11], y la evidencia de campo a gran escala muestra que la tasa de continuación ante una advertencia varía según la severidad percibida y la claridad del mensaje [2]. Un veredicto correcto que no se entiende no modifica la conducta y, por lo tanto, no protege.

8.5. Diseño de las mediciones en hardware

8.5.1. Prueba 1.4: tiempo de escaneo y tasa de detección

La prueba 1.4 constituye el punto de decisión del bloque 1 y mide tres indicadores con definiciones operativas explícitas.

La tasa de detección (TDR) se define como el número de códigos detectados dividido entre el número de códigos presentados, multiplicado por cien, con criterio de aceptación de al menos el 95 %. El tiempo de escaneo (T_{scan}) se mide desde el instante en que la pantalla del escáner queda lista hasta que la cámara entrega un código, con criterio de aceptación de 5 s o menos en el percentil 90; excluye deliberadamente el análisis posterior, que constituye un indicador distinto (T_{F1}) medido contra el backend. La precisión de clasificación se define como la proporción de códigos que abren la pantalla correcta según su tipo de contenido, con criterio de al menos el 98 %.

La muestra consta de 30 códigos que varían las tres condiciones que el uso real impone, conforme al cuadro 8.4. La variación por ángulo es la que más interesa, pues es donde típicamente se degrada la detección; la condición de oscuridad con linterna encendida cumple además la función de verificar el control de linterna sobre hardware real.

Tabla 8.4: Composición de la muestra de la prueba 1.4

Condición	**Niveles**
Tamaño	3 pequeños (∼2 cm, etiqueta o ticket); 4 medianos (∼5 cm, volante o menú); 3 grandes (∼15 cm o más, cartel a distancia)
Ángulo	4 de frente; 3 inclinados ∼30°; 3 inclinados ∼45°
Iluminación	4 con luz de día; 3 en penumbra; 3 en oscuridad con linterna encendida
Contenido	Al menos 20 con URL; 5 de credenciales Wi-Fi; 5 de texto plano o contacto

Los últimos diez códigos, de contenido no navegable, son los que validan la métrica de clasificación: lo que se comprueba en ellos es la lectura por cámara, no el análisis sintáctico del contenido, que ya se verifica en el nivel 1.

La prueba se ejecuta sobre un teléfono Android de gama media y no sobre el mejor dispositivo disponible, dado que el criterio de tiempo de arranque está definido para gama media y conviene medir sobre hardware representativo. La instrumentación reside en la propia aplicación: cada lectura registra su duración y el resumen calcula la mediana, el percentil 90, el máximo, la tasa de detección y el cumplimiento de ambos criterios. El percentil se obtiene por interpolación lineal. Si un código no se lee en 10 s se registra como fallo y se continúa con el siguiente; ese fallo cuenta contra la tasa de detección.

Se eligió el percentil 90 y no el promedio ni el máximo por una razón que conviene dejar asentada: una única lectura lenta no debe descalificar la aplicación, pero tampoco debe poder esconderse en un promedio. Con nueve lecturas de 500 ms y una de 20 s el percentil 90 sigue cumpliendo el criterio; hacen falta dos lecturas lentas para desplazarlo.

Corresponde declarar una limitación del instrumento. Las lecturas exitosas se registran solas, pero los fallos deben anotarse manualmente, porque la aplicación no tiene forma de saber que hubo un código frente a la cámara que nunca llegó a decodificar. Si los fallos no se anotan, la tasa de detección resulta del 100 % y el dato es falso. Es el único valor que el instrumento no puede deducir y por eso el protocolo lo advierte de forma destacada. Adicionalmente, las muestras residen en memoria y se pierden al recargar la aplicación, de modo que los tiempos se anotan conforme se producen.

8.5.2. Protocolo de verificación en dispositivo

El segundo protocolo es una lista de verificación de aproximadamente 50 minutos, pensada para ejecutarse en una sola sesión, organizada en cinco bloques con resultado esperado por paso. Cubre el arranque en frío y el onboarding completo; el análisis real contra el backend, incluyendo los tres veredictos, el comportamiento ante una URL no vista antes y el aviso de espera prolongada; la geocodificación inversa, con criterio de al menos el 85 % de nombres de lugar legibles, que debe ejecutarse al aire libre; la generación de la Story Card, con criterio de al menos el 90 %; el camino sin conexión; y el comportamiento del teclado y las áreas seguras de la pantalla.

Dos pasos del protocolo están marcados como no omisibles. El primero es de seguridad: consiste en intentar escanear con un segundo teléfono el código QR que la Story Card muestra cubierto; si resulta legible, se trata de un fallo grave, y es la razón por la que el código se cubre con una capa opaca y no con un difuminado, pues un código borroso sigue siendo escaneable. El segundo verifica que la preferencia de ubicación sobreviva al reinicio, regresión que se detectó y corrigió durante el desarrollo.

La instrucción transversal del protocolo es registrar qué paso falló y qué se observó, en lugar de marcar únicamente aprobado o reprobado: un paso fallido acompañado del dato de lo ocurrido vale más que diez marcados en verde.

8.6. Latencia del backend y política de degradación

El tiempo límite de la petición de análisis se fijó en 25 s a partir de mediciones sucesivas contra el backend real, que se consignan porque explican una decisión que de otro modo parecería contradecir el criterio de 5 s del indicador T_{F1}. En la primera medición, la llamada inicial tomó 5,5 s por arranque en frío de la función sin servidor y entre 1,6 y 3,1 s en las siguientes. Diez días después, una URL no vista antes por el backend tomaba entre 14,2 y 14,5 s, frente a 0,3 s si ya estaba en caché; el salto coincide con la incorporación del filtro de vista previa visual, que abre un navegador sin interfaz y cuesta alrededor de 10 s. Una medición posterior confirmó 15,3 s en una URL nueva.

Con un límite de 12 s, todo escaneo de un sitio nuevo se interrumpía solo y degradaba a veredicto incierto, es decir, precisamente el caso normal, porque un código QR de interés apunta casi siempre a algo que el backend no ha visto antes. El límite de 25 s deja cerca de 11 s de margen sobre el peor caso medido, y la pantalla de carga avisa al usuario a los 8 s que la espera se está prolongando.

El argumento metodológico que sostiene esta decisión conviene explicitarlo: el tiempo límite no es la métrica. El indicador T_{F1} mide lo que tarda el backend en responder, no el momento en que la aplicación se rinde. Recortar el límite no mejora el número medido; solo convierte análisis buenos en inciertos. Si la latencia del backend disminuye, el límite deberá medirse y reducirse de nuevo, pues 25 s de espera es mucho pedir a una persona que sostiene el teléfono en alto.

8.6.1. Taxonomía de errores y degradación

El cliente HTTP traduce toda condición de fallo a un vocabulario cerrado de once códigos, de modo que la interfaz decida siempre según el código y nunca interpretando el texto del mensaje. La traducción absorbe dos formas de cuerpo de respuesta incompatibles —la acordada en el contrato y la que el endpoint devuelve en la práctica— y normaliza los nombres alternativos que el backend emplea. El cuadro 8.5 resume el criterio de reintento, que la interfaz usa para decidir si ofrece volver a intentar.

Tabla 8.5: Clasificación de errores según posibilidad de reintento

Categoría	Códigos y tratamiento
Reintentable	Tiempo agotado del filtro, filtro no disponible, límite de tasa excedido, error interno, fallo de red y tiempo agotado de la petición
No reintentable	URL inválida, carga inválida y análisis no encontrado: el fallo está en la petición, de modo que repetirla produciría el mismo resultado
Cancelación	Se distingue del tiempo agotado aunque el mecanismo subyacente sea el mismo, pues abandonar la pantalla no es un error y no debe reportarse como tal

La política de degradación distingue dos casos. Una URL inválida se repropaga al llamador, porque no constituye un fallo de análisis sino contenido no analizable, y la pantalla lo trata como tal dirigiendo al usuario a la vista de contenido. Cualquier otro error degrada a un resultado honesto: veredicto incierto con aviso visible de que no se completaron todas las verificaciones. En ningún caso un fallo produce un veredicto seguro, criterio que se deriva de la regla de que la falta de información nunca se comunica en verde.

8.7. Determinación del veredicto

El veredicto se resuelve con una precedencia de dos fuentes. Si el backend concluyó, su veredicto manda; un valor pendiente no constituye conclusión, mientras que un incierto explícito sí se respeta. Solo si el backend no concluyó se deriva el veredicto del puntaje numérico.

Esta derivación existe por una razón circunstancial que conviene documentar: el backend calcula correctamente el puntaje y sus criterios, pero su campo de veredicto final responde pendiente a toda consulta, incluidos dominios legítimos, porque el umbral que convierte puntaje en veredicto no está conectado de su lado. Sin la derivación, todo análisis terminaría en incierto y la aplicación no podría comunicar lo único que debe comunicar. No se trata de un veredicto inventado: el puntaje y los criterios que lo componen provienen del backend; lo único que aporta la aplicación son los cortes. El diseño se desmantela por sí solo, pues en cuanto el backend devuelva veredictos reales la primera fuente tiene precedencia y la derivación deja de ejecutarse sin cambiar una línea.

Los cortes están alineados con los pesos que el propio backend asigna a cada criterio —dominio de nivel superior de riesgo, 25; host que es una dirección IP literal, 30; ausencia de HTTPS, 15; palabra clave sospechosa, 15— según el cuadro 8.6.

Tabla 8.6: Bandas de puntaje y su fundamento

Puntaje	Veredicto	Fundamento
0–29	Seguro	Un solo criterio menor no basta. Marcar de riesgo todo sitio sin HTTPS produciría tantos falsos positivos que la aplicación perdería credibilidad
30–59	Sospechoso	30 es exactamente el peso de un host que es una IP literal, y también la suma de dos criterios menores. Merece advertencia, no condena
60–100	Malicioso	Tres o más señales simultáneas, por ejemplo IP más ausencia de HTTPS más la palabra *login*: el caso clásico de phishing

8.7.1. Validación de rango

El puntaje se valida en dos puntos independientes del flujo, uno para la derivación del veredicto y otro para el medidor visual de riesgo. La duplicación es intencional y ambas validaciones fallan hacia el lado seguro.

Un valor ausente o que no es un número finito no produce veredicto optimista, sino ninguno, y el llamador lo resuelve como incierto. Un valor fuera del rango de 0 a 100 también se rechaza, y por la misma razón de fondo: un -20 o un 150 no significan "muy seguro" y "muy malicioso", sino que el backend cambió de escala y que los cortes ya no significan lo que se supone. Interpretarlos con la tabla anterior es exactamente el mecanismo por el cual se produce un veredicto equivocado con apariencia de correcto, dado que un -20 daría seguro, y un falso seguro es el peor resultado posible del producto. Ante una escala que no se comprende, la aplicación informa incierto, que es la afirmación verdadera.

8.8. Métricas e instrumentos de evaluación

El cuadro 8.7 asocia cada métrica del anteproyecto con el instrumento que la medirá. La columna de instrumento es la aportación metodológica del cuadro: sin ella, una métrica puede enunciarse sin que exista procedimiento capaz de producirla.

Tabla 8.7: Métricas del proyecto y su instrumento de medición

Métrica	Criterio	Instrumento
Clasificación URL / no-URL	$\geq 98\,\%$	Pruebas unitarias y prueba 1.4
Cobertura de pruebas unitarias	$\geq 70\,\%$	Informe de cobertura automatizado
Persistencia offline del historial	$100\,\%$	Pruebas unitarias y protocolo en dispositivo
Reportes con carga completa	$100\,\%$	Pruebas unitarias
T_{scan} (percentil 90)	≤ 5 s	Prueba 1.4 en dispositivo
Tasa de detección (TDR)	$\geq 95\,\%$	Prueba 1.4 en dispositivo
T_{F1} (percentil 90)	≤ 5 s	Medición contra el backend
T_{total}	≤ 35 s	Medición contra el backend
Detección de maliciosos	$\geq 90\,\%$	Muestra de URLs verificadas, requiere backend
Falsos positivos	$\leq 5\,\%$	Muestra de dominios legítimos, requiere backend
Geocodificación legible	$\geq 85\,\%$	Protocolo en dispositivo, bloque B
Story Card generada	$\geq 90\,\%$	Protocolo en dispositivo, bloque C
Comprensión del veredicto	$\geq 90\,\%$	Prueba de usabilidad
Finalización del flujo	$\geq 85\,\%$	Prueba de usabilidad
T_{learn}	≤ 60 s	Prueba de usabilidad
T_{start}	≤ 3 s	Medición en dispositivo de gama media
Compatibilidad Android ≥ 10	Sin errores críticos	Compilación nativa y protocolo en dispositivo

Cuatro de las diecisiete métricas quedan cubiertas por el nivel de pruebas automatizadas. Las trece restantes dependen de hardware, del avance del módulo de backend o de la disponibilidad de participantes, y así se declaran.

Un caso exige precisión particular. La métrica de detección de maliciosos no es medible en el estado actual del sistema, y no por una limitación del instrumento: el backend no devuelve el veredicto malicioso en ninguna consulta, de modo que la medición arrojaría 0 % por construcción y no por desempeño. La métrica queda condicionada a que el módulo de backend conecte su umbral de decisión, y su valor no debe interpretarse hasta entonces. Las métricas de clasificación que sí dependen del frontend se evalúan con las definiciones estándar de precisión y exhaustividad para clasificadores [37].

8.9. Consideraciones éticas y de privacidad

La minimización de datos se trata aquí como decisión de método y no como añadido, pues determina qué puede medirse y cómo.

La regla de mayor jerarquía del producto es que la aplicación nunca visita la URL analizada. Esta regla domina sobre cualquier otra decisión de producto y tiene consecuencia metodológica directa: el análisis opera sobre la cadena de texto, de modo que ninguna medición de este trabajo implica

contacto con la infraestructura del atacante, ni expone la dirección IP del dispositivo, ni revela que un usuario concreto está por acceder a una URL concreta. La vista previa visual, cuando el backend la provee, se presenta como imagen estática y nunca como contenido navegable.

De esa regla se derivan las demás decisiones. La geocodificación inversa se resuelve en el dispositivo y no en el servidor, de modo que las coordenadas no salen del teléfono. Los reportes de códigos maliciosos son anónimos: no incluyen identificador de usuario ni coordenadas, y los campos opcionales solo se transmiten cuando contienen datos. El historial reside exclusivamente en el teléfono, que es la fuente de verdad, lo que además es la condición que hace posible la métrica de recuperación sin conexión. Las contraseñas de redes Wi-Fi codificadas en un QR se enmascaran en pantalla. El código QR que aparece en la Story Card se cubre con una capa opaca y no con un difuminado, porque un código borroso sigue siendo escaneable y compartir la tarjeta propagaría el ataque.

El marco de verificación de seguridad de la aplicación es el estándar de verificación de seguridad de aplicaciones móviles de OWASP en su versión 2.1 [28], del que aplican tres categorías al prototipo: almacenamiento, para el historial local; red, para la comunicación con el backend; y plataforma, para el manejo de permisos de cámara y ubicación. Se eligió por ofrecer criterios verificables en lugar de buenas prácticas generales.

Para las pruebas de usabilidad se recabará consentimiento informado, se explicará el propósito del estudio y el carácter voluntario de la participación, y los datos se tratarán de forma anónima y agregada. Los participantes escanearán únicamente códigos preparados para la prueba, nunca códigos maliciosos reales en entornos de uso cotidiano.

8.10. Limitaciones del método

Corresponde declarar las limitaciones del diseño, con la misma precisión con que se declararon las de la revisión de antecedentes.

La cobertura automatizada alcanza la lógica de negocio pero no las pantallas: no hay pruebas de componentes ni pruebas de extremo a extremo automatizadas, de modo que el comportamiento de la interfaz se verifica mediante protocolos manuales en dispositivo, cuya reproducibilidad depende de quien los ejecute. Tampoco hay analizador estático de estilo configurado: la puerta de calidad del proyecto son la verificación de tipos en modo estricto y el conjunto de pruebas.

La plataforma iOS queda fuera de alcance por falta de hardware, conforme a lo declarado en el capítulo de alcance, de modo que las conclusiones sobre desempeño del escaneo se limitan a Android.

La muestra de las pruebas de usabilidad es de cinco participantes, insuficiente para inferencia estadística. Sus resultados se interpretarán como indicios de problemas de comprensión y no como estimaciones poblacionales, y el estudio de usabilidad con muestra representativa ya figura entre las exclusiones explícitas del trabajo.

Las mediciones de latencia del backend corresponden a un endpoint en ambiente de desarrollo con recursos del nivel gratuito, de modo que los tiempos observados incluyen arranques en frío que un despliegue en producción podría no presentar. El filtro de vista previa visual se comporta de forma intermitente, por lo que no se cuenta con él para las mediciones.

Finalmente, a la fecha de corte de este informe la aplicación no se ha ejecutado en un teléfono físico: no hubo dispositivo ni emulador disponible en el entorno de desarrollo. Lo verificado hasta ahora es que la verificación de tipos está limpia, que las 249 pruebas pasan, que el paquete JavaScript de la aplicación completa se exporta sin errores para Android —lo que garantiza ausencia de importaciones rotas, rutas mal declaradas y errores de sintaxis en las doce pantallas, pero nada de lo que solo se observa ejecutando— y que los tres veredictos se producen correctamente al procesar

respuestas en vivo del backend real. Todo lo que requiere hardware permanece pendiente de los protocolos descritos en la sección 8.5, y así se consigna en lugar de presentarse como verificado.

CAPÍTULO 9

Conclusiones

CAPÍTULO 10

Recomendaciones

Bibliografía

[1] abuse.ch: *URLhaus — API*. URLhaus Documentation. https://urlhaus.abuse.ch/api/.

[2] Akhawe, Devdatta y Adrienne Porter Felt: *Alice in Warningland: A large-scale field study of browser security warning effectiveness*. En *Proceedings of the 22nd USENIX Security Symposium*, páginas 257–272, Washington, D.C., USA, 2013. https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/akhawe.

[3] Amazon Web Services: *AWS Lambda — Developer Guide*. Amazon Web Services Documentation. https://docs.aws.amazon.com/lambda/latest/dg/welcome.html.

[4] Anti-Phishing Working Group: *Phishing Activity Trends Report, 1st Quarter 2024*. Informe técnico, APWG, 2024. https://docs.apwg.org/reports/apwg_trends_report_q1_2024.pdf.

[5] Anti-Phishing Working Group: *Phishing Activity Trends Report, 1st Quarter 2025*. Informe técnico, APWG, 2025. https://docs.apwg.org/reports/apwg_trends_report_q1_2025.pdf.

[6] Apple Inc.: *CLGeocoder — Core Location*. Apple Developer Documentation. https://developer.apple.com/documentation/corelocation/clgeocoder.

[7] Apple Inc.: *NSAppTransportSecurity — Information Property List*. Apple Developer Documentation. https://developer.apple.com/documentation/bundleresources/information-property-list/nsapptransportsecurity.

[8] Apple Inc.: *Requesting authorization to use location services*. Apple Developer Documentation. https://developer.apple.com/documentation/corelocation/requesting-authorization-to-use-location-services.

[9] Apple Inc.: *VNDetectBarcodesRequest — Vision framework*. Apple Developer Documentation. https://developer.apple.com/documentation/vision/vndetectbarcodesrequest.

[10] Apple Inc.: *Apple Platform Security*. Informe técnico, Apple Inc., 2024. https://help.apple.com/pdf/security/en_US/apple-platform-security-guide.pdf.

[11] Cranor, Lorrie Faith: *A framework for reasoning about the human in the loop*. En *Proceedings of the 1st Conference on Usability, Psychology, and Security (UPSEC '08)*, página 1, San Francisco, CA, USA, 2008. https://www.usenix.org/legacy/event/upsec08/tech/full_papers/cranor/cranor.pdf.

[12] Egelman, Serge, Lorrie Faith Cranor y Jason Hong: *You've been warned: An empirical study of the effectiveness of web browser phishing warnings*. En *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '08)*, páginas 1065–1074, Florence, Italy, 2008.

[13] Google: *Google Safe Browsing — Overview (v5).* Google for Developers. `https://developers.google.com/safe-browsing`.

[14] Google: *Network security configuration.* Android Developers. `https://developer.android.com/privacy-and-security/security-config`.

[15] Google: *Request location permissions.* Android Developers. `https://developer.android.com/develop/sensors-and-location/location/permissions`.

[16] Google: *Scan barcodes with ML Kit on Android.* Google for Developers. `https://developers.google.com/ml-kit/vision/barcode-scanning/android`.

[17] Google LLC: *VirusTotal API v3 — Overview.* VirusTotal Documentation. `https://docs.virustotal.com/reference/overview`.

[18] Insikt Group: *Security Challenges Rise as QR Code and AI-Generated Phishing Proliferate.* Informe técnico CTA-2024-0718, Recorded Future, Julio 2024. `https://www.recordedfuture.com/research/qr-code-and-ai-generated-phishing-proliferate`.

[19] International Organization for Standardization: *Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*, 3rd edición, 2015. `https://www.iso.org/standard/62021.html`.

[20] International Organization for Standardization: *Information technology — Automatic identification and data capture techniques — QR code bar code symbology specification*, 4th edición, 2024. `https://www.iso.org/standard/83389.html`.

[21] Krombholz, Katharina, Peter Frühwirt, Peter Kieseberg, Ioannis Kapsalis, Markus Huber y Edgar Weippl: *QR code security: A survey of attacks and challenges for usable security.* En Tryfonas, Theo y Ioannis Askoxylakis (editores): *Human Aspects of Information Security, Privacy, and Trust (HAS 2014)*, volumen 8533 de *Lecture Notes in Computer Science*, páginas 79–90, Cham, Switzerland, 2014. Springer.

[22] Le, Anh, Athina Markopoulou y Michalis Faloutsos: *PhishDef: URL names say it all.* En *Proceedings of IEEE INFOCOM 2011*, páginas 191–195, Shanghai, China, 2011. `https://arxiv.org/abs/1009.2275`.

[23] Ma, Justin, Lawrence K. Saul, Stefan Savage y Geoffrey M. Voelker: *Beyond blacklists: Learning to detect malicious web sites from suspicious URLs.* En *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '09)*, páginas 1245–1254, Paris, France, 2009.

[24] Mapbox: *Geocoding API — Mapbox Search Service.* Mapbox API Documentation. `https://docs.mapbox.com/api/search/geocoding/`.

[25] Meta Platforms: *About the New Architecture.* React Native Documentation. `https://reactnative.dev/architecture/landing-page`.

[26] MISP Project: *MISP — Open Source Threat Intelligence Platform & Open Standards For Threat Information Sharing.* MISP Project. `https://www.misp-project.org/`.

[27] OpenPhish: *OpenPhish — Phishing Intelligence.* OpenPhish. `https://openphish.com/`.

[28] OWASP Foundation: *OWASP Mobile Application Security Verification Standard (MASVS) v2.1.* OWASP Mobile Application Security, 2024. `https://mas.owasp.org/MASVS/`.

[29] Palo Alto Networks Unit 42: *Phishing on the edge of the web and mobile using QR codes.* Unit 42 Threat Research, 2023. `https://unit42.paloaltonetworks.com/qr-codes-as-attack-vector/`.

[30] PhishTank / Cisco Talos: *Developer Information.* PhishTank. `https://phishtank.org/developer_info.php`.

[31] Rousavy, Marc: *react-native-mmkv — The fastest key/value storage for React Native.* GitHub Repository. `https://github.com/mrousavy/react-native-mmkv`.

[32] Rousavy, Marc: *StorageBenchmark — Benchmark App to compare different storage libraries (MMKV, AsyncStorage, WatermelonDB, RealmDB, SQLite).* GitHub Repository. `https://github.com/mrousavy/StorageBenchmark`.

[33] Rousavy, Marc: *VisionCamera — The Barcode Scanner.* react-native-vision-camera Documentation. `https://react-native-vision-camera.com/docs/guides/code-scanning`.

[34] Sahoo, Doyen, Chenghao Liu y Steven C. H. Hoi: *Malicious URL detection using machine learning: A survey.* arXiv preprint arXiv:1701.07179, 2017. `https://arxiv.org/abs/1701.07179`.

[35] Salahdine, Fatima y Naima Kaabouch: *Social Engineering Attacks: A Survey.* Future Internet, 11(4):89, Abril 2019. `https://www.mdpi.com/1999-5903/11/4/89`.

[36] Sheng, Steve, Brad Wardman, Gary Warner, Lorrie Faith Cranor, Jason Hong y Chengshan Zhang: *An empirical analysis of phishing blacklists.* En *Proceedings of the 6th Conference on Email and Anti-Spam (CEAS 2009)*, Mountain View, CA, USA, 2009. `https://kilthub.cmu.edu/articles/journal_contribution/An_Empirical_Analysis_of_Phishing_Blacklists/6469805`.

[37] Sokolova, Marina y Guy Lapalme: *A systematic analysis of performance measures for classification tasks.* Information Processing & Management, 45(4):427–437, Julio 2009.

[38] Verizon: *2024 Data Breach Investigations Report.* Informe técnico, Verizon Business, 2024. `https://www.verizon.com/business/resources/reports/2024-dbir-data-breach-investigations-report.pdf`.

[39] Vidas, Timothy, Emmanuel Owusu, Shuai Wang, Cheng Zeng, Lorrie Faith Cranor y Nicolas Christin: *QRishing: The susceptibility of smartphone users to QR code phishing attacks.* En Adams, Andrew A., Michael Brenner y Matthew Smith (editores): *Financial Cryptography and Data Security (FC 2013 Workshops)*, volumen 7862 de *Lecture Notes in Computer Science*, páginas 52–69, Berlin, Germany, 2013. Springer.

ANEXO A

Planos de Construcción